Declaración de uso de IA Entrega 3

Vicente Vial:

IA usada: ChatGPT.

Alcance y contexto:

Usé la IA de ChatGPT como apoyo para entender y desarrollar distintas partes del proyecto Guerra del Pacífico incluyendo la configuración del backend con el archivo .env, el inicio de PostgreSQL, la creación de la base de datos, la ejecución de migraciones, la organización según MVC y el CRUD de usuarios con separación de rutas, controladores, servicios y modelos de Sequelize, validaciones y manejo de errores además de la implementación del registro y login, el reemplazo de mocks por peticiones HTTP y el mantenimiento del estado del usuario y las rutas protegidas en React, también lo usé para que me explicara como enviar y recuperar el despliegue en la preparación de partidas, validar el tablero y aplicar filtros de pertenencia al consultar, modificar y eliminar acciones. También lo usé como apoyo para configurar peticiones, headers, cuerpos JSON, variables y comprobaciones de respuestas en Postman además de exportar las colecciones al repositorio.
El apoyo de ChatGPT para esta entrega incluye explicaciones paso a paso y ejemplos de código para las funcionalidades mencionadas.

Aporte y Análisis Crítico:

Usé ChatGPT como apoyo para entender cómo organizar el backend y conectar las vistas de React con PostgreSQL y a partir de sus explicaciones y ejemplos adapté las rutas, los campos y las funciones a la estructura del proyecto separando las responsabilidades entre controladores, servicios y modelos. En el CRUD de usuarios ajusté los ejemplos para trabajar con los campos username, email, password y role e incorporé las validaciones de campos obligatorios, formato de correo y registros duplicados, además de respuestas que excluyen la contraseña. También adapté el manejo de errores al formato “codigo” y “mensaje” para que el frontend pudiera mostrar el problema correspondiente.

En Login y Register reemplacé las comprobaciones con datos mock por peticiones al backend y ajusté los cuerpos JSON para que coincidieran con lo esperado por los endpoints y conecté sus respuestas con la navegación al menú. También incorporé la conservación de los datos públicos del usuario en sessionStorage y la redirección a login desde las páginas que requieren una sesión, además la comprobación del código de administrador la dejé en el servidor usando una variable de entorno.

En Preparación adapté el envío de los barcos al formato del backend y la recuperación de las posiciones guardadas e integré la consulta periódica del estado, el tiempo restante informado por el servidor y el bloqueo de la edición después de confirmar.

Las validaciones se ajustaron a las reglas del juego que son cuatro barcos de tipos distintos, sin posiciones repetidas y dentro de la zona del jugador. En acciones adapté las consultas para relacionar cada registro con su usuario y partida restringiendo la modificación y eliminación del historial a partidas terminadas y para documentar los endpoints preparé las peticiones de Postman con las rutas, headers, variables y cuerpos utilizados por el proyecto

En general la IA me sirvió como referencia para entender conceptos y obtener ejemplos iniciales pero revisé el código, lo modifiqué y lo integré para ajustarlo al diseño, la navegación y los datos usados en el proyecto.

Tomás Mancilla
IA usada: Claude
Alcance y contexto:
Usé Claude como apoyo para entender y desarrollar distintas partes del proyecto. En primer lugar, la IA me ayudó con la estructura inicial del servidor en Koa, con la separación en rutas, controladores, servicios y middlewares, la configuración de CORS y el middleware global de manejo de errores. También lo usé para trabajar con Sequelize en los modelos, migraciones, asociaciones, validaciones y seeders de las partidas y de la relación entre usuarios y partidas. También para las peticiones del frontend al servidor desde el menú de React y sus handlers, incluyendo crear partida, unirse por código, buscar rival y los estados del lobby. Por último, usé la IA como ayuda para implementar el término de los turnos, abandonar partida y los eventos de cada ronda.

Aporte y Análisis Crítico:

Para la estructura inicial de Koa, Claude me orientó sobre el orden en que deben ir los middlewares y sobre cómo centralizar el manejo de errores. A partir de eso ajusté el middleware para que todas las respuestas de error siguieran el formato “código” y “mensaje” del protocolo definido en la Entrega 2, de modo que el frontend pudiera mostrar siempre el problema correspondiente.

En Sequelize me apoyé en Claude para entender cómo modelar las relaciones entre usuarios y partidas mediante la tabla JugadorPartida, y adapté sus campos a lo definido en

el modelo ER y en las reglas del juego, como los recursos iniciales, el rol dentro de la partida y el número de jugador.

En el frontend, Claude me ayudó con los handlers para crear partida, unirse por código y buscar rival, e incorporé consultas periódicas para el lobby y el emparejamiento.

Para los turnos y eventos, Claude me ayudó a implementar el cambio de turno, el abandono y las cartas de evento basadas en las reglas de la propuesta del juego, las cuales revisé, probé en mi entorno e integré al proyecto.

En general, la IA me sirvió como apoyo para entender conceptos, revisar alternativas, obtener ejemplos de código y detectar errores, pero siempre revisé lo que me entregaba, lo probé y lo integré ajustándolo al diseño del proyecto y a las reglas del juego.

Miguel Mujica
IA usada: Claude Code

Alcance y contexto:

Usé Claude Code durante mi parte de la entrega, que fue el tablero, los barcos y los recursos, donde la idea fue que la ia sea el motor de todo lo que es más mecanico y yo ser el que decide sobre ella, guiando paso a paso en conjunto para entender todo el codigo, yo le iba indicando paso a paso qué hacer y la herramienta escribía el código, lo probaba en mi entorno y luego preparaba los commits y los textos de los PR, los cuales revisaba antes de subirlos. En primer lugar me ayudó a dejar funcionando el entorno local, ya que tenía dos PostgreSQL instalados, uno en Windows y otro en Ubuntu, y mi usuario existía solo en el de Ubuntu. Luego lo usé para crear con Sequelize los modelos, migraciones, asociaciones, validaciones y seeders del catálogo de mejoras, las casillas, los barcos, las casillas reveladas, las mejoras de cada barco y las acciones, además de las constantes de las reglas y las funciones del tablero, como generar el mapa con sus 12 casillas especiales, calcular distancias y lo que ve cada barco. También me apoyé en la IA para generar el tablero cuando la partida pasa a despliegue, el endpoint para desplegar la flota con su plazo de 2 minutos, el estado de la partida filtrado por jugador para mantener la niebla de guerra y la acción de mover, junto con el seeder de la partida demo, la colección de Postman de mis endpoints y mi sección del README. Por último, la usé para conectar la pantalla de Partida del frontend con el backend, incluyendo los botones de terminar turno y abandonar, para resolver un conflicto de merge con un cambio de Tomás y para redactar los mensajes al

grupo.

Aporte y Análisis Crítico:

Mi trabajo fue principalmente dirigir y revisar lo que hacía la herramienta, ya que el orden de los pasos lo definí yo en base a lo que necesitaba de Tomás y Vicente, y antes de programar acordé con ellos los puntos que unían nuestras áreas, como dejar los recursos dentro de JugadorPartida, identificar al jugador con el header X-Usuario-Id, agregar el numeroJugador para saber la zona de despliegue, usar ids numéricos y tener un único GET /games/:id, lo cual evitó tener que rehacer modelos después.

En las reglas que no estaban claras en la propuesta, yo decidía como abordar lo que no estaba tan claro, por ejemplo que una corriente peligrosa pueda quedar dentro de una zona de despliegue sin hacer daño, ya que la regla 10 solo daña al barco que entra moviéndose, y que el radio de detección se mida en distancia Manhattan igual que el movimiento y el alcance. Lo mismo pasó con el plazo de 2 minutos del despliegue, donde elegí revisarlo cada vez que se consulta la partida en vez de usar temporizadores, con crear la tabla Acciones para controlar el máximo de 2 acciones por barco y con responder 422 en las jugadas inválidas, como pide la Entrega 4.

Sobre lo que generaba le pedí varios cambios, como que no agregara dependencias nuevas. Cada paso lo probé antes de subirlo con peticiones HTTP sobre los usuarios demo y en el navegador, donde por ejemplo apareció que una clave foránea con RESTRICT terminaba en un error 500, por lo que la cambiamos a NO ACTION. Cuando mis compañeros subieron cambios, como el seeder y los turnos de Tomás, revisé cómo afectaban mi parte y ajusté mis ramas antes de abrir los PR.

Además, entre cada paso le hacía preguntas para entender lo que se estaba haciendo como si lo estubiese haciendo yo para si entender a la perfeccion lo desarrollado y poder hacer los cambios que quiera a gusto, como para qué sirven el up y el down de una migración, por qué hay validaciones en el modelo y también restricciones en la base de datos, la diferencia entre hasMany, belongsTo y belongsToMany o por qué el servidor nunca le manda el estado completo al cliente, ya que quería quedar con todo claro de lo que estaba haciendo.

Capturas de pantalla

1. Iniciar el proyecto e instalar dependencias

```
npm init -y
```

```
npm install koa @koa/router koa-bodyparser @koa/cors sequelize pg dotenv
```

```
npm install --save-dev sequelize-cli nodemon
```

En `package.json`, cambia `"main"` a `"src/index.js"`.

2. Estructura de carpetas

Primero crea el archivo `.sequelizerc` en la raíz. Le indica a `sequelize-cli` dónde dejar cada cosa, así todo queda dentro de `src/`:

```
const path = require('path');

module.exports = {
  config: path.resolve('src', 'config', 'database.js'),
  'models-path': path.resolve('src', 'models'),
  'migrations-path': path.resolve('src', 'migrations'),
  'seeders-path': path.resolve('src', 'seeders'),
};
```

4. Middleware global de errores

Usa el mismo formato de error del protocolo JSON (`{ codigo, mensaje }`), así se cumple el criterio de consistencia con E2.

src/utils/HttpError.js: es el error que lanzarán todos los servicios, como `throw new HttpError(404, 'PARTIDA_NO_ENCONTRADA', '...')`.

```
class HttpError extends Error {
  constructor(status, codigo, mensaje) {
    super(mensaje);
    this.status = status;
    this.codigo = codigo;
  }
}

module.exports = HttpError;
```

src/middlewares/errorHandler.js:

```
const { ValidationError, UniqueConstraintError } = require('sequelize');

module.exports = async (ctx, next) => {
  try {
    await next();
    if (ctx.status === 404 && !ctx.body) {
      ctx.status = 404;
      ctx.body = { codigo: 'RUTA_NO_ENCONTRADA', mensaje: `No existe ${ctx.method}
${ctx.path}` };
    }
  } catch (err) {
    if (err instanceof UniqueConstraintError) {
      ctx.status = 409;
      ctx.body = { codigo: 'REGISTRO_DUPLICADO', mensaje: err.errors.map((e) =>
e.message).join('; ') };
    } else if (err instanceof ValidationError) {
      ctx.status = 400;
      ctx.body = { codigo: 'VALIDACION_FALLIDA', mensaje: err.errors.map((e) =>
e.message).join('; ') };
    } else if (err.codigo) {
```

5. Rutas y controlador de prueba

src/controllers/health.controller.js: además de responder, verifica que la base de datos esté conectada.

```
const { sequelize } = require('../models');

async function getHealth(ctx) {
  await sequelize.authenticate();
  ctx.body = { estado: 'OK', baseDeDatos: 'conectada' };
}

module.exports = { getHealth };
```

src/routes/health.routes.js:

```
const Router = require('@koa/router');
const { getHealth } = require('../controllers/health.controller');

const router = new Router();
router.get('/health', getHealth);

module.exports = router;
```

src/routes/index.js junta todas las rutas. Cada compañero agregará aquí sus archivos, por ejemplo tú `games.routes.js`:

```
const Router = require('@koa/router');
const healthRoutes = require('./health.routes');

const router = new Router();
router.use(healthRoutes.routes());

module.exports = router;
```

```
async function request(path, options = {}) {
  const response = await fetch(`${BASE_URL}${path}`, {
    headers: {
      "Content-Type": "application/json",
      ...options.headers,
    },
    ...options,
  });

  if (!response.ok) {
    throw new Error(`Error ${response.status}: ${response.statusText}`);
  }

  return response.json();
}

export function getUsers() {
  return request("/users", { method: "GE` `);
}
```

```
  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

- `useState`: guarda los datos, el estado de carga y el error.
- `useEffect` con `[]`: ejecuta la llamada una sola vez, cuando el componente se monta.
- `.then / .catch / .finally`: manejan éxito, fallo y limpieza (quitar el "cargando") respectivamente.

3. El lado del backend

El backend solo necesita exponer endpoints que respondan JSON. Ejemplo mínimo en Express (Node):

3. El lado del backend

El backend solo necesita exponer endpoints que respondan JSON. Ejemplo mínimo en Express (Node):

javascript

```javascript
app.get("/api/users", (req, res) => {
  res.json([{ id: 1, name: "Ana" }]);
});
```

O en FastAPI (Python):

python

```python
@app.get("/api/users")
def get_users():
    return [{"id": 1, "name": "Ana"}]
```

como usar sequelize en el backend

Explicando cómo integrar Sequelize en el backend 6s

Sequelize es un ORM para Node.js: en vez de escribir SQL a mano, defines modelos (clases/objetos JS que representan tablas) y usas métodos como `findAll` o `create`. Funciona con PostgreSQL, MySQL, MariaDB, SQLite y SQL Server.

1. Instalación

bash

```bash
npm install sequelize
npm install pg pg-hstore   # driver para PostgreSQL
# MySQL: npm install mysql2
# SQLite: npm install sqlite3
```

Sequelize necesita el driver de tu base de datos por separado.

2. Conexión

javascript

```javascript
// src/config/database.js
const { Sequelize } = require("sequelize");

const sequelize = new Sequelize("mi_base", "mi_usuario", "mi_password", {
  host: "localhost",
  dialect: "postgres",
  logging: false,
});

module.exports = sequelize;
```

- `new Sequelize(base, usuario, password, opciones)`: crea la conexión.
- `dialect`: el tipo de base de datos (`postgres`, `mysql`, `sqlite`...).
- `logging: false`: evita que imprima cada SQL generado en consola (ponlo en `console.log` si quieres verlos al depurar).

3. Definir un modelo

javascript

```javascript
// src/models/User.js
const { DataTypes } = require("sequelize");
const sequelize = require("../config/database");

const User = sequelize.define(
  "User",
  {
    id: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
    },
    name: {
      type: DataTypes.STRING,
      allowNull: false,
    },
    email: {
      type: DataTypes.STRING,
      allowNull: false,
```

```
        isEmail: true,
      },
    },
  },
  {
    tableName: "users",
    timestamps: true,
  }
);

module.exports = User;
```

- `sequelize.define(nombreModelo, columnas, opciones)`: crea el modelo.
- `DataTypes`: tipos de columna (`STRING`, `INTEGER`, `BOOLEAN`, `DATE`, `TEXT`, `FLOAT`...).
- `allowNull: false`: la columna es obligatoria (`NOT NULL`).
- `unique: true`: no permite valores repetidos.
- `validate`: validaciones que Sequelize ejecuta antes de guardar.
- `tableName`: nombre exacto de la tabla (si no lo pones, pluraliza el nombre del modelo).
- `timestamps: true`: agrega automáticamente `createdAt` y `updatedAt`.

4. Sincronizar con la base de datos

javascript

```
// src/server.js
const sequelize = require("./config/database");
require("./models/User");

sequelize
  .sync()
  .then(() => console.log("Tablas sincronizadas"))
  .catch((error) => console.error(error));
```

- `sync()`: crea las tablas si no existen.
- `sync({ alter: true })`: ajusta las tablas existentes al modelo.
- `sync({ force: true })`: borra y recrea las tablas (solo para desarrollo).

2. Usarlo en un componente

javascript

```
import { useState, useEffect } from "react";
import { getUsers } from "./services/api";

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    getUsers()
      .then((data) => setUsers(data))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>Cargando...</p>;
  if (error) return <p>Error: {error}</p
```

¿Cómo preparo el entorno del backend? Guíame paso a paso para configurar el `.env`, iniciar PostgreSQL y crear la base de datos.

Ha trabajado durante 31s >

Para tu backend de **Guerra del Pacífico**, que utiliza **Node.js, Koa, Sequelize y PostgreSQL**, sigue estos pasos desde la terminal de Ubuntu en WSL.

1. Entra al proyecto e instala sus dependencias

```bash
cd ~/Web/AlexisGoatWEB-Backend-S2-26-2
npm install
```

`npm install` descarga las dependencias declaradas en `package.json`. Para comprobar que tienes Node y npm disponibles:

```bash
node -v
npm -v
```

2. Crea y configura el `.env`

En la raíz del proyecto, donde está `package.json`, copia la configuración de ejemplo:

```bash
cp .env.example .env
```

Si ya tienes un `.env` configurado, edítalo directamente, porque ese comando lo sobrescribe.

El `.env` guarda la configuración que cambia entre computadores, como las credenciales de la base de datos. Tu aplicación debe cargar esas variables para poder utilizarlas. Node.js v26.12.1 ...

Para una instalación local nueva, puedes usar este ejemplo:

```dotenv
PORT=3000
DB_HOST=localhost
DB_PORT=5432
DB_USER=guerra_user
DB_PASSWORD=TU_CLAVE_LOCAL
DB_NAME=guerra_pacifico_dev
FRONTEND_URL=http://localhost:5173
```

Variable	Qué representa
`PORT`	Puerto donde escucha el backend.
`DB_HOST`	Dirección del servidor de PostgreSQL.
`DB_PORT`	Puerto de PostgreSQL, habitualmente `5432`.
`DB_USER`	Usuario de PostgreSQL que utilizará el backend.
`DB_PASSWORD`	Contraseña de ese usuario.
`DB_NAME`	Nombre de la base de datos del proyecto.
`FRONTEND_URL`	Dirección del frontend para la configuración de CORS.

Reemplaza `TU_CLAVE_LOCAL` por una contraseña propia. Si ya tienes un usuario de PostgreSQL para el proyecto, conserva sus credenciales en lugar de crear otro.

Comprueba también que `.gitignore` incluya:

```gitignore
.env
```

Trabaja con ChatGPT

GPT-6.1 Sol Alto

Reemplaza `TU_CLAVE_LOCAL` por una contraseña propia. Si ya tienes un usuario de PostgreSQL para el proyecto, conserva sus credenciales en lugar de crear otro.

Comprueba también que `.gitignore` incluya:

```gitignore
.env
node_modules/
```

El `.env.example` puede subirse al repositorio con valores de ejemplo y sin contraseñas reales.

3. Inicia PostgreSQL

Si todavía no está instalado en Ubuntu:

```bash
sudo apt update
sudo apt install postgresql postgresql-contrib
```

Luego inicia el servicio y comprueba su estado:

```bash
sudo service postgresql start
sudo service postgresql status
```

La contraseña que pide `sudo` **es la de tu usuario de Ubuntu.** PostgreSQL tiene sus propios usuarios y contraseñas. Ubuntu Server d...

4. Prepara el usuario de la base de datos

Abre la consola de PostgreSQL como administrador:

```bash
sudo -u postgres psql
```

Ahora estarás dentro de `psql`; el indicador normalmente será parecido a `postgres=#`. Revisa los usuarios existentes:

```sql
\du
```

Si estás siguiendo el ejemplo y `guerra_user` no existe, créalo:

```sql
CREATE ROLE guerra_user WITH LOGIN CREATEDB;
```

`LOGIN` permite conectarse y `CREATEDB` permite crear la base mediante Sequelize. postgresql.org

Asigna su contraseña:

```
\password guerra_user
```

Escribe la misma contraseña que pusiste en `DB_PASSWORD`. Luego sal:

```
\q
```

Si el usuario ya existía, no repitas `CREATE ROLE`. Si necesitas darle permiso para crear la base, ejecuta dentro de `psql`:

```sql
ALTER ROLE guerra_user CREATEDB;
```

Trabaja con ChatGPT

+ GPT-6.1 Sol Alto

Si el usuario ya existía, no repitas `CREATE ROLE`. Si necesitas darle permiso para crear la base, ejecuta dentro de `psql`:

```sql
ALTER ROLE guerra_user CREATEDB;
```

5. Comprueba la conexión

Desde la terminal de Ubuntu:

```bash
psql -h localhost -p 5432 -U guerra_user -d postgres -W
```

Introduce la contraseña de PostgreSQL. Nos conectamos inicialmente a `postgres` porque la base del proyecto todavía no está creada.

Si entras correctamente, puedes comprobar la conexión y salir:

```
\conninfo
\q
```

6. Crea la base y ejecuta las migraciones

Desde la raíz del proyecto, los comandos que habíamos utilizado son:

```bash
npm run db:create
npm run db:migrate
```

Ejecuta primero uno y, cuando termine correctamente, el siguiente.

- `db:create` crea la base configurada en el proyecto.
- `db:migrate` ejecuta las migraciones pendientes para crear las tablas, columnas y relaciones. Sequelize registra las migraciones ejecutadas en `SequelizeMeta`. Sequelize

Si la base ya existe, continúa con las migraciones. **No necesitas borrarla ni recrearla.**

Para revisar las tablas:

```bash
psql -h localhost -p 5432 -U guerra_user -d guerra_pacifico_dev -W
```

Dentro de `psql`:

```
\dt
\q
```

7. Inicia el backend

Consulta los scripts disponibles:

```bash
npm run
```

Si aparece el script `dev`, ejecútalo:

```bash
npm run dev
```

Deja esa terminal abierta mientras utilizas el backend.

Los errores más habituales se interpretan así:

Error	Qué re
`Connection refused`	PostgreSQL está detenido o el host/puerto es incorrecto.

Trabaja con ChatGPT

GPT-6.1 Sol Alto

Bash

Deja esa terminal abierta mientras utilizas el backend.

Los errores más habituales se interpretan así:

Error	Qué revisar
`Connection refused`	PostgreSQL está detenido o el host/puerto es incorrecto.
`password authentication failed`	`DB_USER` o `DB_PASSWORD` no coincide con PostgreSQL.
`database ... does not exist`	Falta crear la base o `DB_NAME` es incorrecto.
`permission denied to create database`	El usuario necesita el permiso `CREATEDB`.
`Missing script`	Revisa `npm run` y los scripts actuales de `package.json`.

La preparación queda comprobada cuando puedes conectarte con las credenciales del `.env`, ves las tablas tras las migraciones y el backend inicia sin errores de conexión.

¿Cómo organizo el backend y construyo el CRUD de usuarios? Explícame MVC, los modelos y migraciones de Sequelize, las validaciones y el manejo de errores.

Ha trabajado durante 24s

En tu proyecto con **Koa**, **Sequelize** y **PostgreSQL**, conviene construir el CRUD de usuarios separando tres cosas: **recibir peticiones HTTP, aplicar las reglas del sistema y consultar la base de datos.** Así puedes modificar una parte sin tener que reescribir las demás.

Los nombres de campos y archivos que usaré son ejemplos; debes ajustarlos a los que ya existen en tu proyecto.

1. Organiza las responsabilidades: MVC y servicios

MVC significa Modelo, Vista y Controlador. En una aplicación con React y una API, React se encarga de la interfaz; el backend entrega datos mediante JSON.

Las rutas y los servicios complementan esa organización:

Parte	Responsabilidad	Ejemplo en usuarios
Rutas	Asociar un método y una URL con un controlador.	`POST /usuarios` llama a `crearUsuario`.
Controladores	Leer la petición y construir la respuesta HTTP.	Obtener el cuerpo y responder con `201`.
Servicios	Aplicar las reglas del sistema y coordinar operaciones.	Normalizar el correo y generar el hash de la contraseña.
Modelos	Representar los datos y permitir consultas con Sequelize.	Crear o buscar un usuario.
Migraciones	Definir los cambios en la estructura de la base de datos.	Crear la tabla `Usuarios`.
Middlewares	Ejecutar tareas comunes alrededor de las peticiones.	Autenticación, validación y manejo de errores.

Por ejemplo, podrías distribuir el trabajo en `routes/usuarios.js`, `controllers/usuariosController.js`, `services/usuariosService.js` y `models/usuario.js`, manteniendo las convenciones actuales del repositorio.

El controlador conoce HTTP; el servicio conoce las reglas del sistema. Evita pasar `ctx` al servicio: pásale los datos que necesita.

2. Define qué información representa un usuario

Antes de programar, establece sus campos y restricci s. Por ejemplo:

Campo	Uso	Restricciones

Trabaja con ChatGPT

GPT-6.1 Sol Alto

repositorio.

El controlador conoce HTTP; el servicio conoce las reglas del sistema. Evita pasar `ctx` al servicio: pásale los datos que necesita.

2. Define qué información representa un usuario

Antes de programar, establece sus campos y restricciones. Por ejemplo:

Campo	Uso	Restricciones
`id`	Identificar al usuario.	Clave primaria autoincremental.
`nombre`	Mostrar su nombre.	Obligatorio y con longitud limitada.
`email`	Identificar su cuenta.	Obligatorio, válido y único.
`passwordHash`	Guardar el resultado del hash de su contraseña.	Obligatorio; se excluye de las respuestas.
`createdAt` y `updatedAt`	Registrar fechas.	Administradas por Sequelize.

El modelo describe estos datos para que Sequelize pueda trabajar con ellos. Una definición parcial podría ser:

JavaScript

```
nombre: {
  type: DataTypes.STRING(80),
  allowNull: false,
  validate: {
    notEmpty: true,
    len: [1, 80]
  }
},
email: {
  type: DataTypes.STRING,
  allowNull: false,
  unique: true,
  validate: {
    isEmail: true
  }
}
```

A partir del modelo puedes ejecutar operaciones como `Usuario.create(...)` o `Usuario.findByPk(id)`. Editar el modelo no actualiza por sí solo una tabla existente cuando trabajas con migraciones. Sequelize

3. Crea la migración que construye la tabla

El modelo permite usar los datos desde JavaScript. La migración modifica su estructura en PostgreSQL.

Una migración tiene dos funciones:

- `up` aplica el cambio: por ejemplo, crea `Usuarios`.
- `down` revierte ese cambio: por ejemplo, elimina esa tabla.

Para una tabla nueva, puedes generar el archivo:

Bash

```
npx sequelize-cli migration:generate --name create-usuarios
```

Después completas `up` con `queryInterface.createTable(...)`, incluyendo las columnas, tipos, clave primaria, restricciones y fechas. `down` utiliza `queryInterface.dropTable(...)`.

Finalmente ejecutas:

Bash

```
npm run db:migrate
```

Sequelize registra qué migraciones ya ejecutó. Si posteriormente agregas un campo a una tabla compartida, crea otra migración para agregarlo; cambiar una migración ya aplicada no hace que vuelva a ejecutarse. Sequelize

Si tu proyecto ya tiene la migración de usuarios, utiliza esa como punto de partida y evita crear la misma tabla otra vez.

4. Define el contrato del CRUD

Trabaja con ChatGPT

GPT-6.1 Sol Alto

Sequelize registra qué migraciones ya ejecutó. Si posteriormente agregas un campo a una tabla compartida, crea otra migración para agregarlo; cambiar una migración ya aplicada no hace que vuelva a ejecutarse. Sequelize

Si tu proyecto ya tiene la migración de usuarios, utiliza esa como punto de partida y evita crear la misma tabla otra vez.

4. Define el contrato del CRUD

CRUD significa crear, leer, actualizar y eliminar. Antes de implementar, decide las rutas, los datos que aceptan y sus respuestas:

Operación	Método y ruta de ejemplo	Operación de Sequelize	Respuesta exitosa
Crear	`POST /usuarios`	`Usuario.create(...)`	`201` y usuario creado.
Listar	`GET /usuarios`	`Usuario.findAll(...)`	`200` y arreglo de usuarios.
Obtener uno	`GET /usuarios/:id`	`Usuario.findByPk(id)`	`200` y usuario.
Actualizar parcialmente	`PATCH /usuarios/:id`	Buscar y luego `usuario.update(...)` .	`200` y usuario actualizado.
Eliminar	`DELETE /usuarios/:id`	Buscar y luego `usuario.destroy()` .	`204` , sin cuerpo.

Para obtener, actualizar o eliminar un usuario, si el identificador es válido pero el registro no existe, responde `404` .

Una lista sin resultados normalmente devuelve `200` con `[]` .

5. Construye cada operación siguiendo el mismo recorrido

Veamos la creación de un usuario.

1. La ruta recibe `POST /usuarios` y llama al controlador.
2. La validación de entrada comprueba que el cuerpo tenga los campos y tipos esperados.
3. El controlador obtiene los datos de `ctx.request.body` y llama al servicio.
4. El servicio normaliza el correo, aplica las reglas y genera el hash de la contraseña.
5. El modelo guarda el registro en PostgreSQL.
6. El controlador responde con `201` y los campos públicos del usuario.

Un controlador pequeño podría tener esta forma:

JavaScript

```
async function crearUsuario(ctx) {
  const usuario = await usuariosService.crear(ctx.request.body);

  ctx.status = 201;
  ctx.body = {
    id: usuario.id,
    nombre: usuario.nombre,
    email: usuario.email
  };
}
```

Este ejemplo supone que la validación y el servicio ya están implementados.

Para actualizar, usa una lista explícita de campos permitidos, como `nombre` y `email` . Evita entregar directamente todo el cuerpo a `update` : alguien podría intentar modificar campos como `id` o `rol` .

La contraseña necesita un tratamiento específico: validar la nueva contraseña y generar un nuevo hash. Nunca almacenes la contraseña original ni devuelvas su hash en el JSON. OWASP recomienda Argon2id para nuevos sistemas. OWASP Cheat S...

6. Valida la entrada y protege la base de datos

Las validaciones cumplen funciones distintas según dónde estén:

Lugar	Qué revisar
Entrada HTTP	Cuerpo válido, tipos, campos obligatorios, longitudes e identificador entero positivo.
Servicio	Reglas del sistema, permisos y campos que pueden modificarse.
Modelo	Validaciones de Sequelize, como formato de correo

Trabaja con ChatGPT

GPT-6.1 Sol Alto

Lugar	Qué revisar
Entrada HTTP	Cuerpo válido, tipos, campos obligatorios, longitudes e identificador entero positivo.
Servicio	Reglas del sistema, permisos y campos que pueden modificarse.
Modelo	Validaciones de Sequelize, como formato de correo.
PostgreSQL	Restricciones definitivas: `NOT NULL`, `UNIQUE` y claves foráneas.

Sequelize distingue las validaciones ejecutadas en JavaScript de las restricciones comprobadas en la base de datos. Necesitas ambas para proteger los datos.

Por ejemplo, consultar previamente si existe un correo permite dar un mensaje claro, pero también debe existir una restricción `UNIQUE`: dos peticiones simultáneas podrían comprobar que el correo está disponible antes de que alguna lo guarde.

Además:

- Al crear, exige todos los campos obligatorios.
- En un `PATCH`, valida los campos presentes y rechaza cambios vacíos.
- Aplica la misma normalización del correo al crear y al actualizar.
- Comprueba autorización: estar autenticado no significa poder modificar cualquier usuario.

7. Centraliza el manejo de errores

En Koa puedes colocar un middleware con `try/catch` antes del parser del cuerpo y de las rutas, para capturar los errores de los middlewares posteriores mediante `await next()`.

Ese middleware debe traducir los errores a respuestas coherentes:

Situación	Estado HTTP
JSON mal formado, identificador inválido o datos incorrectos.	`400`
Falta autenticación válida.	`401`
El usuario no tiene permiso.	`403`
El usuario solicitado no existe.	`404`
Correo duplicado.	`409`
Eliminación impedida por una relación existente.	`409`
Fallo inesperado del servidor.	`500`

Puedes usar un formato común:

```JSON
{
  "error": {
    "code": "EMAIL_DUPLICADO",
    "message": "Ese correo ya está registrado."
  }
}
```

Distingue especialmente `SequelizeUniqueConstraintError` de `SequelizeValidationError`, porque representan problemas diferentes. Para fallos inesperados, registra el detalle en el servidor y entrega al cliente un mensaje genérico, sin consultas SQL, credenciales ni trazas internas.

8. Comprueba el CRUD en Postman

Prueba una secuencia completa:

1. Crear un usuario válido y guardar su `id`.
2. Consultarlo por ese `id`.
3. Actualizar un campo y volver a consultarlo.
4. Intentar crear otro con el mismo correo.
5. Enviar datos inválidos y comprobar el error.
6. Eliminarlo y verificar que una consulta posterior responda `404`.
7. Intentar modificar un usuario sin los permisos necesarios.

Trabaja con ChatGPT

GPT-6.1 Sol Alto

1. Crear un usuario válido y guardar su `id`.
2. Consultarlo por ese `id`.
3. Actualizar un campo y volver a consultarlo.
4. Intentar crear otro con el mismo correo.
5. Enviar datos inválidos y comprobar el error.
6. Eliminarlo y verificar que una consulta posterior responda `404`.
7. Intentar modificar un usuario sin los permisos necesarios.

Comprueba también que ninguna respuesta incluya la contraseña ni su hash. Así verificas tanto las operaciones del CRUD como sus validaciones y errores.

¿Cómo implemento y conecto el registro y login? Explícame la validación de credenciales y del código de administrador, cómo reemplazar los mocks y cómo mantener la sesión y proteger las rutas en React.

Ha trabajado durante 51s >

Para conectarlos, el backend debe decidir si el registro o el login son válidos; React envía los datos, muestra el resultado y actualiza el estado del usuario.

En el flujo que veníamos trabajando, usabas `POST /auth/register` y `POST /auth/login`, respuestas con `{ usuario }` y `sessionStorage` para conservar el usuario al recargar. JWT quedaba para una etapa posterior. Esa persistencia permite mantener la interfaz, pero todavía no acredita la identidad ante el backend.

1. Define qué recibe y devuelve cada endpoint

Mantén un contrato claro entre frontend y backend:

Endpoint	Datos enviados, como ejemplo	Respuesta exitosa
`POST /auth/register`	`username`, `email`, `password` y, si corresponde, `codigoAdmin`.	`201` con `{ usuario }`.
`POST /auth/login`	`email` y `password`.	`200` con `{ usuario }`.

Estos nombres deben coincidir exactamente con los que recibe tu backend. Si el login utiliza `username`, ambos lados deben utilizarlo.

Una respuesta puede ser:

JSON

```json
{
  "usuario": {
    "id": 2,
    "username": "jugador_auth",
    "email": "auth@uc.cl",
    "role": "Jugador"
  }
}
```

La respuesta nunca incluye la contraseña ni su hash. El rol lo determina el servidor.

2. Implementa las validaciones del registro

El recorrido sería:

1. La ruta llama al controlador de registro.
2. El controlador obtiene los datos y llama al servicio.
3. El servicio valida los datos, determina el rol y genera el hash.
4. Sequelize guarda el usuario.
5. El controlador devuelve `201`.

Valida en el backend:

- Que los campos obligatorios existan y tengan el tipo correcto.
- Que `username` cumpla las longitudes permitidas.
- Que el correo tenga formato válido.
- Que la contraseña cumpla la política definida.
- Que correo y nombre de usuario no estén repeti[illegible], si ambos deben ser únicos.

Normaliza el correo de forma consistente, por ejemplo eliminando espacios alrededor y convirtiéndolo a

Trabaja con ChatGPT

GPT-6.1 Sol Alto

Valida en el backend:

- Que los campos obligatorios existan y tengan el tipo correcto.
- Que `username` cumpla las longitudes permitidas.
- Que el correo tenga formato válido.
- Que la contraseña cumpla la política definida.
- Que correo y nombre de usuario no estén repetidos, si ambos deben ser únicos.

Normaliza el correo de forma consistente, por ejemplo eliminando espacios alrededor y convirtiéndolo a minúsculas. **No recortes ni transformes silenciosamente la contraseña.**

React puede comprobar estos requisitos antes de enviar para ayudar al usuario, pero el backend debe repetirlos: una petición puede enviarse sin pasar por tu formulario.

La contraseña se convierte en un hash mediante una biblioteca apropiada. Para verificarla posteriormente se utiliza la función de esa biblioteca; no se compara con texto plano. OWASP recomienda Argon2id para sistemas nuevos. OWASP Cheat S...

3. Valida el código de administrador exclusivamente en el backend

No tengo confirmada la regla exacta de tu código. Si su función es permitir registrarse como administrador, puedes definir este comportamiento:

Situación	Resultado
Registro sin código.	Crear un jugador.
Registro con código correcto.	Crear un administrador.
Registro con código incorrecto.	Rechazar el registro.
Código enviado, pero configuración del servidor ausente.	Rechazar la creación de administrador.

El código esperado debe estar en el `.env` del backend, por ejemplo bajo el nombre `ADMIN_REGISTRATION_CODE`.

El servicio comprueba el código y asigna el rol. Evita guardar directamente un `role` enviado por el cliente:

JavaScript

```javascript
// Esquema de la decisión, con nombres ilustrativos.
let role = "Jugador";

if (solicitaRegistroAdmin) {
  validarCodigoAdmin(codigoAdmin);
  role = ROL_ADMIN;
}
```

`validarCodigoAdmin` debe rechazar valores vacíos, tipos incorrectos y códigos que no coincidan. No guardes el código en el registro del usuario ni lo incluyas en respuestas o logs.

No uses `VITE_ADMIN_CODE`: las variables `VITE_` quedan expuestas en el código del navegador. Vite

Una vez creada la cuenta, el login obtiene el rol guardado en la base de datos. El código de registro no tiene que volver a enviarse en cada inicio de sesión, salvo que hayas definido expresamente otro mecanismo.

4. Implementa el login y la comprobación de credenciales

El servicio de login debe:

1. Validar los tipos y la presencia del identificador y la contraseña.
2. Buscar al usuario por el campo acordado.
3. Verificar la contraseña contra el hash almacenado.
4. Si coincide, devolver únicamente sus datos públicos.
5. Si falla, devolver `401`.

Para un usuario inexistente y una contraseña incorrecta, utiliza el mismo mensaje:

> Correo o contraseña incorrectos.

Así evitas revelar qué cuentas existen. También conviene limitar los intentos repetidos en el servidor. GitHub

Distingue esos casos de un cuerpo inválido (`400`) o un fallo inesperado (`500`). El frontend debe mostrar el error recibido y conservar el formulario para corregirlo.

5. Reemplaza los mocks por peticiones reales

En el frontend, configura la dirección del backend:

Trabaja con ChatGPT

+ GPT-6.1 Sol Alto

GitHub

Distingue esos casos de un cuerpo inválido (`400`) o un fallo inesperado (`500`). El frontend debe mostrar el error recibido y conservar el formulario para corregirlo.

5. Reemplaza los mocks por peticiones reales

En el frontend, configura la dirección del backend:

dotenv

```
VITE_API_URL=http://localhost:3000
```

Reinicia Vite después de cambiar el `.env` . Esta dirección puede ser pública; las credenciales de PostgreSQL y el código administrativo pertenecen al backend. Vite

Centraliza las peticiones en un servicio, por ejemplo:

JavaScript

```
async function enviarAuth(endpoint, datos) {
  const respuesta = await fetch(`${API_URL}/auth/${endpoint}`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(datos)
  });

  const resultado = await respuesta.json();

  if (!respuesta.ok) {
    throw new Error(
      resultado.error?.message ?? "No se pudo completar la solicitud."
    );
  }

  return resultado.usuario;
}

export const login = (datos) => enviarAuth("login", datos);
export const registrar = (datos) => enviarAuth("register", datos);
```

Este ejemplo supone respuestas JSON y el formato de error anterior. `fetch` no lanza una excepción automáticamente por recibir `400` o `401` , por eso debes comprobar `respuesta.ok` .

En los formularios, reemplaza:

Comportamiento del mock	Comportamiento conectado
Buscar credenciales en `usuarios.json` .	Llamar a `/auth/login` .
Agregar usuarios a un arreglo local.	Llamar a `/auth/register` .
Asignar un rol desde React.	Usar el rol devuelto por el servidor.
Navegar inmediatamente al menú.	Navegar después de una respuesta exitosa.

Mientras esperas, deshabilita el botón y muestra un estado de carga. Si falla la petición, muestra el error; evita volver al mock como alternativa.

Configura también CORS en Koa para permitir el origen del frontend, por ejemplo `http://localhost:5173` .

6. Comparte y conserva el estado del usuario en React

Un `AuthContext` puede exponer:

Valor o función	Propósito
`usuario`	Usuario público actual o `null` .
`iniciarSesion(usuario)`	Actualizar React y guardar el usuario.
`cerrarSesion()`	Limpiar React y el almacenamiento.
`cargando`	Indicar que se está restaurando o verificando el acceso.

Context permite que el menú, las páginas y las rutas lean el mismo estado sin pasar propiedades por todos los componentes. React

Trabaja con ChatGPT

+ GPT-6.1 Sol Alto

`cerrarSesion()`	Limpiar React y el almacenamiento.
`cargando`	Indicar que se está restaurando o verificando el acceso.

Context permite que el menú, las páginas y las rutas lean el mismo estado sin pasar propiedades por todos los componentes. React

Para el comportamiento que tenías con `sessionStorage`:

- Al iniciar sesión, guarda solamente el usuario público.
- Al arrancar React, lee ese dato usando `try/catch`; si está corrupto, elimínalo.
- Al cerrar sesión, elimina el dato y establece `usuario = null`.

Esto conserva la interfaz al recargar en la misma pestaña. **No guardes contraseñas, códigos administrativos ni credenciales de sesión en ese almacenamiento.** El contenido puede modificarse desde el navegador y no debe utilizarse como prueba de identidad o autorización. OWASP Cheat S...

7. Protege las rutas de React

Puedes envolver `/menu`, `/preparacion` y `/tablero` con un componente que consulte el contexto:

```JavaScript
import { Navigate, Outlet } from "react-router-dom";

function RutaProtegida() {
  const { usuario, cargando } = useAuth();

  if (cargando) return <p>Cargando...</p>;

  return usuario
    ? <Outlet />
    : <Navigate to="/login" replace />;
}
```

`useAuth` representa un hook que tú defines para leer `AuthContext`. `Outlet` muestra la página hija de esa ruta. React Router

Después agrupas las páginas:

```JavaScript
<Route element={<RutaProtegida />}>
  <Route path="/menu" element={<Menu />} />
  <Route path="/preparacion" element={<Preparacion />} />
  <Route path="/tablero" element={<Tablero />} />
</Route>
```

Para una página administrativa, comprueba además el rol y muestra acceso denegado si no corresponde.

Estos controles organizan la navegación. Los permisos sobre datos y acciones también deben comprobarse en cada endpoint del backend.

8. Completa la sesión autenticada cuando conectes esa etapa

Con respuestas que solo contienen `{ usuario }`, el servidor no puede reconocer de forma confiable las siguientes peticiones. Enviar un `usuarioId` no demuestra quién está haciendo la solicitud.

Para completar ese mecanismo, una opción es una sesión del servidor identificada mediante una cookie:

1. El login válido crea una sesión nueva.
2. El servidor entrega una cookie `HttpOnly`, con `Secure` en producción y una política `SameSite` adecuada.
3. El navegador envía la cookie en las peticiones correspondientes.
4. Un middleware identifica al usuario y comprueba sus permisos.
5. Al recargar, React consulta un endpoint como `/auth/me`.
6. Al cerrar sesión, el backend invalida la sesión y React limpia su estado.

Con frontend y API en orígenes distintos, las peticiones con cookies necesitan `credentials: "include"` y CORS con origen explícito y credenciales habilitadas. Las operaciones que modifican datos requieren protección frente a CSRF. OWASP Cheat S...

Para verificar la integración, prueba registro normal, código administrativo correcto e incorrecto, credenciales inválidas, recarga, cierre de sesión y acceso directo a una ruta protegida. Cuando incorpores la sesión del servidor, comprueba también que alterar el usuario guardado en el navegador no permita ejecutar acciones de otro usuario o administrador.

Trabaja con ChatGPT

GPT-6.1 Sol Alto

¿Cómo conecto la preparación de la partida y completo el CRUD de acciones? Guíame para enviar el despliegue, recuperar su estado y limitar la consulta y modificación del historial al usuario correspondiente.

Ha trabajado durante 47s >

Para conectarlo, necesitas que React envíe el despliegue al backend, recupere el estado guardado y consulte únicamente las acciones del usuario correspondiente. Las reglas de pertenencia y de estado de la partida deben comprobarse en el servidor.

Tomaré como referencia las rutas que habíamos trabajado. Los ejemplos muestran el procedimiento; los nombres concretos deben coincidir con tus modelos actuales.

1. Define las peticiones de preparación y acciones

Operación	Método y ruta
Recuperar el estado de una partida	`GET /games/:partidaId`
Enviar el despliegue	`POST /games/:partidaId/desplegar`
Crear una acción	`POST /games/:partidaId/acciones`
Consultar el historial propio	`GET /games/:partidaId/acciones`
Consultar una acción propia	`GET /games/:partidaId/acciones/:accionId`
Modificar una acción propia	`PATCH /games/:partidaId/acciones/:accionId`
Eliminar una acción propia	`DELETE /games/:partidaId/acciones/:accionId`

En lo que habíamos planteado, la modificación y eliminación del historial se permiten únicamente cuando la partida está `TERMINADA`.

2. Conecta el botón de despliegue en React

En `Preparacion.jsx`, mantén las posiciones seleccionadas como estado local mientras el jugador acomoda sus barcos.

Cuando presione "Confirmar despliegue":

1. Comprueba que haya colocado todos los barcos.
2. Construye el cuerpo que espera el backend.
3. Envía la petición.
4. Actualiza la interfaz con el resultado del servidor.

El formato que habíamos utilizado era:

JavaScript

```
const datos = {
  barcos: barcosColocados.map((barco) => ({
    tipo: barco.tipo,
    casilla: {
      x: barco.x,
      y: barco.y
    }
  }))
};
```

Para el mecanismo provisional de E3:

JavaScript

```
const respuesta = await fetch(
  `${import.meta.env.VITE_API_URL}/games/${partidaId}/desplegar`,
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "X-Usuario-Id": String(usuario.id)
    },
    body: JSON.stringify(datos)
  }
```

Trabaja con ChatGPT

GPT-6.1 Sol Alto

```javascript
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "X-Usuario-Id": String(usuario.id)
    },
    body: JSON.stringify(datos)
  }
);

const resultado = await respuesta.json();

if (!respuesta.ok) {
  throw new Error(
    resultado.error?.message ?? "No se pudo guardar el despliegue."
  );
}
```

Mientras se envía, deshabilita el botón. Si falla, conserva las posiciones para que el usuario pueda corregirlas o reintentar.

No marques al jugador como listo antes de recibir una respuesta exitosa.

3. Valida y guarda el despliegue en el backend

La distribución de responsabilidades puede ser:

Capa	Trabajo
Ruta	Asociar la URL con el controlador.
Middleware	Obtener la identidad del solicitante y validar parámetros.
Controlador	Leer el cuerpo y llamar al servicio.
Servicio	Comprobar participación, reglas del tablero y estado de la partida.
Modelos	Guardar barcos, posiciones y estado de preparación.

El servicio debe comprobar:

- Que la partida exista y el usuario participe en ella.
- Que siga en una fase que permita desplegar.
- Que el jugador no haya confirmado previamente, si no permites reemplazar el despliegue.
- Que estén todos los barcos exigidos, con tipos y cantidades válidos.
- Que las coordenadas sean enteras y estén dentro del tablero.
- Que las posiciones pertenezcan a la zona inicial del jugador.
- Que no haya superposiciones ni posiciones prohibidas por las reglas.

Para un tablero de 10×10 con índices desde cero, los límites serían `0` a `9`; frontend y backend deben utilizar la misma convención.

El backend debe obtener del usuario su zona inicial. No debe aceptar que el cliente elija la zona o el participante al que pertenece el despliegue.

Guarda los cambios en una transacción:

1. Bloquea la fila de la partida para coordinar despliegues simultáneos.
2. Vuelve a comprobar su estado y la preparación del jugador.
3. Guarda todos sus barcos.
4. Marca su preparación como completa.
5. Comprueba si ambos jugadores están listos y realiza la transición correspondiente.
6. Confirma los cambios.

Así evitas guardar algunos barcos y fallar después. En una transacción administrada de Sequelize, un error provoca la reversión de los cambios; todas las consultas implicadas deben recibir la misma transacción.
Sequelize

4. Recupera el estado al entrar o recargar

Al montar la preparación, consulta:

```
GET /games/:partidaId
```

Utiliza la respuesta para restaurar:

- El estado actual de la partida.

Trabaja con ChatGPT

GPT-6.1 Sol Alto

Al montar la preparación, consulta:

```
Texto sin formato
GET /games/:partidaId
```

Utiliza la respuesta para restaurar:

- El estado actual de la partida.
- Los barcos y posiciones propios.
- Si el usuario ya confirmó.
- Si el rival está listo, cuando ese dato sea público.

El backend debe preparar una respuesta acorde con lo que ese jugador puede ver. **No envíes las posiciones ocultas del rival para luego esconderlas en React.**

Mientras cargas, muestra un estado de espera. Si el usuario ya había desplegado, reconstruye el tablero y bloquea la edición según las reglas. Si la partida avanzó, navega a la pantalla correspondiente.

Si todavía no tienes WebSocket, puedes consultar periódicamente mientras esperas al rival. Detén esa consulta al salir de la pantalla o cambiar de fase.

5. Identifica al usuario antes de consultar su historial

Aquí hay una distinción necesaria: `X-Usuario-Id` permite probar el flujo de E3, pero cualquiera puede cambiar esa cabecera. Validar que ese usuario existe no demuestra que sea quien envió la petición.

Para limitar realmente el acceso, un middleware debe obtener la identidad desde una sesión o un token verificado. Luego el servicio utiliza esa identidad, por ejemplo:

```javascript
const usuarioId = ctx.state.usuario.id;
```

Ese valor supone que un middleware de autenticación ya estableció `ctx.state.usuario`.

En ningún caso permitas que el cuerpo de la petición sustituya la identidad utilizada para autorizar. Los controles de pertenencia deben ejecutarse en cada operación del servidor. GitHub

6. Aplica la pertenencia a todo el CRUD de acciones

Para listar, filtra simultáneamente por partida y propietario:

```javascript
const acciones = await Accion.findAll({
  where: {
    partidaId,
    usuarioId
  },
  order: [["createdAt", "ASC"], ["id", "ASC"]]
});
```

Antes, comprueba que el usuario tenga acceso a esa partida.

Para consultar, modificar o eliminar una acción concreta, agrega su identificador:

```javascript
const accion = await Accion.findOne({
  where: {
    id: accionId,
    partidaId,
    usuarioId
  }
});
```

Las tres condiciones deben cumplirse a la vez. Buscar únicamente por `accionId` permitiría intentar acceder a registros de otro jugador. Sequelize permite expresar estos filtros mediante `where`. Sequelize

Después aplica las reglas de cada operación:

Operación	Comprobaciones
Crear	Participación, tipo de acción válido, datos permitidos y fase correspondiente.
Listar	Participación y filtro por propietario.
Obtener una	Coincidencia de acción, partida y propietario.

Trabaja con ChatGPT

GPT-6.1 Sol Alto

Modificar	Pertenencia, partida `TERMINADA` y campos editables válidos.
Eliminar	Pertenencia y partida `TERMINADA`.

Al crear, asigna `usuarioId` desde la identidad del solicitante y `partidaId` desde la ruta validada. Evita copiar todo el cuerpo directamente al modelo.

En `PATCH`, construye una lista explícita de campos editables. Rechaza cambios de propietario, partida, identificador o campos calculados por el servidor. También rechaza actualizaciones vacías.

Modificar o eliminar una entrada del historial no debe volver a ejecutar ni deshacer una jugada. Si el juego depende de esos registros para reconstruir su estado, necesitas separar el historial editable de los eventos utilizados por la lógica de juego.

7. Refleja los cambios reales en React

Reemplaza los arreglos simulados por el resultado de `GET /games/:partidaId/acciones`.

Después de una modificación exitosa, reemplaza la entrada con la respuesta del servidor o vuelve a consultar el listado. Después de una eliminación exitosa, retírala del estado local o recarga el historial.

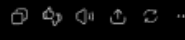
Puedes ocultar los botones de editar y eliminar hasta que la partida termine, pero el backend debe mantener la misma restricción aunque alguien envíe una petición manualmente.

8. Verifica el flujo con dos usuarios

En la colección de Postman que veníamos preparando, utiliza una partida y acciones de dos usuarios diferentes:

Prueba	Resultado esperado
Despliegue válido	Se guarda completo y se recupera al recargar.
Despliegue inválido	Se rechaza sin guardar parcialmente.
Usuario ajeno a la partida	Acceso rechazado.
Historial consultado por A	Solo aparecen acciones de A.
A utiliza el identificador de una acción de B	No puede consultarla, modificarla ni eliminarla.
Acción correcta con otra `partidaId`	No se encuentra dentro de ese ámbito.
Modificación antes de `TERMINADA`	Se rechaza, por ejemplo con `409`.
Modificación propia válida después del término	Se guarda y aparece al consultar nuevamente.
Intento de cambiar `usuarioId`	Se rechaza.

Con autenticación implementada, agrega una prueba clave: **cambiar `X-Usuario-Id` no debe cambiar la identidad reconocida por el servidor.** Esa comprobación diferencia el filtrado funcional de E3 de una restricción real de acceso.



¿Cómo pruebo y documento mis endpoints en Postman? Explícame cómo configurar las peticiones, headers, cuerpos y variables, comprobar las respuestas y exportar las colecciones al repositorio.

Ha trabajado durante 51s >

Postman te permite probar el backend directamente: **eliges el endpoint, envías datos y compruebas tanto la respuesta como el cambio guardado en la base de datos.** Después puedes exportar esas peticiones para que otra persona reproduzca las pruebas.

1. Prepara el backend

Antes de abrir las pruebas:

- Inicia PostgreSQL.
- Ejecuta las migraciones pendientes.
- Inicia el backend.
- Comprueba el puerto configurado en su `.env`.

Trabaja con ChatGPT

+ GPT-6.1 Sol Alto

1. Prepara el backend

Antes de abrir las pruebas:

- Inicia PostgreSQL.
- Ejecuta las migraciones pendientes.
- Inicia el backend.
- Comprueba el puerto configurado en su `.env`.

Si escucha en el puerto `3000`, la dirección base será:

```
Texto sin formato
http://localhost:3000
```

Usa una base de desarrollo con datos de prueba, porque ejecutarás operaciones que crean, modifican y eliminan registros.

2. Crea la colección y sus variables

En **Collections**, crea una colección llamada, por ejemplo, **Guerra del Pacífico**. Puedes organizarla en carpetas: `Autenticación`, `Usuarios`, `Preparación` y `Acciones`.

Después, en **Environments**, crea un entorno llamado **Local** con estas variables:

Variable	Valor
`baseUrl`	`http://localhost:3000`
`usuarioId`	ID de un usuario de prueba existente.
`usuarioAjenoId`	ID de otro usuario.
`partidaId`	ID de una partida de prueba.
`accionId`	ID de una acción propia.
`emailPrueba`	Correo de la cuenta de prueba.
`passwordPrueba`	Contraseña de esa cuenta.

Selecciona **Local** como entorno activo. Para utilizar una variable, escribe su nombre entre llaves dobles:

```
Texto sin formato
{{baseUrl}}/games/{{partidaId}}/acciones
```

Postman reemplazará esas referencias antes de enviar la petición. Evita definir la misma variable también en la colección, porque el alcance puede cambiar qué valor se utiliza. Postman Docs

3. Configura método, URL y headers

Crea una petición para consultar el historial propio:

```
Texto sin formato
GET {{baseUrl}}/games/{{partidaId}}/acciones
```

En **Headers**, para el mecanismo provisional de E3, agrega:

Key	Value
`X-Usuario-Id`	`{{usuarioId}}`
`Accept`	`application/json`

Un `GET` de este tipo no necesita cuerpo.

Para las peticiones que envían JSON, utiliza además:

```
Texto sin formato
Content-Type: application/json
```

`X-Usuario-Id` corresponde al flujo provisional que trabajamos; no autentica realmente al usuario.

Trabaja con ChatGPT

+ GPT-6.1 Sol Alto

Para las peticiones que envían JSON, utiliza además:

```
Texto sin formato
Content-Type: application/json
```

`X-Usuario-Id` corresponde al flujo provisional que trabajamos; no autentica realmente al usuario. Cuando implementes JWT, configurarás Authorization → Bearer Token; si utilizas sesiones con cookies, probarás ese mecanismo. En registro y login no necesitas enviar esta cabecera de usuario.

4. Configura los cuerpos de las peticiones

Para probar el login:

```
Texto sin formato
POST {{baseUrl}}/auth/login
```

En Body, selecciona raw → JSON:

```
JSON
{
  "email": "{{emailPrueba}}",
  "password": "{{passwordPrueba}}"
}
```

Para registro, siguiendo los campos que habíamos utilizado:

```
Texto sin formato
POST {{baseUrl}}/auth/register
```

```
JSON
{
  "username": "jugador_postman",
  "email": "{{emailPrueba}}",
  "password": "{{passwordPrueba}}"
}
```

Seleccionar JSON hace que Postman agregue el encabezado correspondiente. Revisa que no tengas otro `Content-Type` contradictorio. Postman Docs

Para preparación y acciones, utiliza los campos exactos que acepta tu backend:

- Despliegue: el arreglo completo de barcos con sus tipos y posiciones.
- Crear acción: los campos requeridos por el servicio de acciones.
- Modificar acción: solamente los campos editables.

El código administrativo, cuando corresponda, se envía con el nombre que valide tu endpoint de registro. Usa un valor local y retíralo de cualquier exportación compartida.

5. Envía y comprueba la respuesta

Presiona **Send**. Revisa tres cosas:

Elemento	Qué debes comprobar
Estado HTTP	Coincide con el resultado esperado.
Cuerpo	Contiene los datos o el error correcto.
Persistencia	Una consulta posterior refleja el cambio.

Por ejemplo:

Caso	Resultado esperado según el contrato trabajado
Registro válido	`201` , usuario público creado.
Login válido	`200` , usuario público.
Contraseña incorrecta	`401` , credenciales inválidas.
Historial propio	`200` , solo acciones del usuario correspondiente.

Trabaja con ChatGPT
+ GPT-6.1 Sol Alto

Caso	Resultado esperado según el contrato trabajado
Registro válido	`201`, usuario público creado.
Login válido	`200`, usuario público.
Contraseña incorrecta	`401`, credenciales inválidas.
Historial propio	`200`, solo acciones del usuario correspondiente.
Modificar historial antes de `TERMINADA`	Rechazo, por ejemplo `409` si así lo implementaste.
Eliminar una acción	El estado definido por tu endpoint; si es `204`, sin cuerpo.

No basta con obtener `200`. Después de un `PATCH`, consulta el registro para verificar el cambio. Después de un `DELETE`, consulta ese identificador y comprueba que ya no esté disponible.

Un error de validación esperado debe producir una respuesta controlada, no un `500`.

6. Agrega comprobaciones automáticas

En la petición, abre **Scripts → Post-response**. En versiones anteriores puede aparecer como **Tests**.

Para un login exitoso con respuesta `{ usuario }`, puedes escribir:

```javascript
pm.test("Login válido y respuesta pública", function () {
  pm.response.to.have.status(200);

  const usuario = pm.response.json().usuario;

  pm.expect(usuario.id).to.be.a("number");
  pm.expect(usuario).to.not.have.property("password");
  pm.expect(usuario).to.not.have.property("passwordHash");

  pm.environment.set("usuarioId", usuario.id);
});
```

Además de comprobar la respuesta, guarda el identificador para las siguientes peticiones. Ajusta los nombres si tu modelo utiliza otros campos sensibles.

En una petición separada de login incorrecto:

```javascript
pm.test("Rechaza credenciales incorrectas", function () {
  pm.response.to.have.status(401);
});
```

Los resultados aparecen en **Test Results**. Los scripts se guardan con las peticiones y pueden ejecutarse al correr la colección. Postman Docs

Cuando captures `partidaId` o `accionId`, utiliza la estructura real de la respuesta. **Guarda variables únicamente después de comprobar que la creación fue exitosa.**

7. Prueba los casos negativos y la pertenencia

Guarda peticiones distintas con nombres claros, por ejemplo:

Nombre de petición	Qué cambia
`Login - contraseña incorrecta`	Envía una contraseña inválida.
`Registro - correo duplicado`	Reutiliza un correo existente.
`Despliegue - posición inválida`	Envía una coordenada fuera del tablero.
`Acciones - consultar propias`	Utiliza `usuarioId`.
`Acciones - consultar acción ajena`	Utiliza el otro usuario contra la acción original.
`Acciones - modificar antes del término`	Usa una partida que todavía no está terminada.
`Acciones - eliminar propia`	Usa una acción propia y una partida terminada.

En el flujo provisional puedes alternar `X-Usuario-Id` para comprobar los filtros. Con autenticación real,

Trabaja con ChatGPT

GPT-6.1 Sol Alto

`Acciones - modificar antes del término`	Usa una partida que todavía no está terminada.
`Acciones - eliminar propia`	Usa una acción propia y una partida terminada.

En el flujo provisional puedes alternar `X-Usuario-Id` para comprobar los filtros. Con autenticación real, debes iniciar sesión como el segundo usuario para verificar los permisos; cambiar una cabecera no sustituye esa prueba.

Ordena las pruebas que dependan unas de otras: crear, consultar, modificar, volver a consultar y eliminar al final. Para repetirlas, documenta cómo regenerar los datos o restablecer el escenario.

8. Documenta cada petición

En la descripción de cada petición, explica:

- Qué hace.
- Qué registros deben existir antes.
- Qué variables, headers y campos requiere.
- Qué respuesta exitosa devuelve.
- Qué errores controlados puede producir.
- Qué restricciones aplica, como pertenencia o partida `TERMINADA`.

Guarda ejemplos de respuestas exitosas y de error mediante la opción de guardar la respuesta como ejemplo. Postman permite asociarlos a las peticiones para ilustrar su uso. Postman Docs

9. Exporta la colección al repositorio

Primero guarda los cambios de todas las peticiones.

En la interfaz actual:

1. Abre **Collections**.
2. Pulsa los tres puntos de la colección.
3. Selecciona **More → Export collection**.
4. Descarga el archivo mediante **Export JSON**.

En versiones anteriores puede aparecer directamente **Export** y ofrecer **Collection v2.1**. El entorno se exporta por separado desde sus propias opciones. Postman Docs

Para la colección de acciones, conserva el nombre que veníamos usando:

Texto sin formato

```
docs/postman/acciones.postman_collection.json
```

Si guardas el entorno, publica una plantilla, por ejemplo:

Texto sin formato

```
docs/postman/local.postman_environment.example.json
```

Antes de agregar los archivos, revisa que no incluyan contraseñas reales, códigos administrativos, tokens o cookies, tampoco dentro de ejemplos de respuesta.

Desde la raíz del repositorio:

Bash

```
git status
git add docs/postman/acciones.postman_collection.json
git diff --cached
git commit -m "docs: agregar colección Postman de acciones"
git push
```

Agrega por su nombre cualquier otro archivo que hayas exportado y revisado.

En el README indica cómo importar la colección y el entorno, iniciar el backend, completar las variables y ejecutar las peticiones en orden. Finalmente, importa la exportación nuevamente y comprueba que funciona siguiendo esas instrucciones.

Una última comprobación debe hacerse desde React: Postman permite probar la API, pero su funcionamiento no confirma que CORS esté correctamente configurado para el navegador.

ChatGPT puede cometer errores. Comprueba la información importante.

Trabaja con ChatGPT

GPT-6.1 Sol Alto

como funciona la migracion up y down ?

Sí, `up` y `down` son las dos mitades de una migración: `up` aplica el cambio a la base de datos y `down` lo deshace exactamente. Juntas permiten avanzar y retroceder el esquema de forma controlada, y que todo el equipo tenga la misma estructura de tablas.

Cómo funcionan

- `up`: se ejecuta con `npx sequelize-cli db:migrate`. Crea tablas, agrega columnas, índices, etc.
- `down`: se ejecuta con `npx sequelize-cli db:migrate:undo` (la última) o `db:migrate:undo:all` (todas). Hace lo contrario de `up`.

Sequelize guarda en una tabla especial llamada `SequelizeMeta` los nombres de las migraciones ya ejecutadas. Así sabe cuáles faltan por correr y cuál fue la última (la que se deshace con `undo`). Los archivos se ejecutan en orden alfabético, por eso llevan un timestamp al inicio (`20260915120000-crear-partidas.js`).

Ejemplo mínimo

javascript

```javascript
// migrations/20260915120000-crear-partidas.js
"use strict";

module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable("Partidas", {
      id: {
        type: Sequelize.INTEGER,
        primaryKey: true,
        autoIncrement: true,
        allowNull: false,
      },
      estado: {
        type: Sequelize.STRING,
        allowNull: false,
        defaultValue: "esperando",
      },
      createdAt: { type: Sequelize.DATE, allowNull: false },
      updatedAt: { type: Sequelize.DATE, allowNull: false },
    });
  },
```

```
  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable("Partidas");
  },
};
```

Explicación línea por línea

- `"use strict"`: activa el modo estricto de JavaScript (evita errores silenciosos).
- `module.exports = { up, down }`: sequelize-cli busca exactamente esas dos funciones en el archivo.
- `queryInterface`: objeto que permite modificar la estructura de la BD (`createTable`, `dropTable`, `addColumn`, `removeColumn`, `addIndex`, etc.). No es el modelo; las migraciones no usan los modelos.
- `Sequelize`: se recibe para usar los tipos (`Sequelize.INTEGER`, `Sequelize.STRING`...).
- `createTable("Partidas", {...})`: crea la tabla con sus columnas. Aquí hay que declarar `createdAt` y `updatedAt` a mano si tu modelo usa `timestamps`.
- `async` y `await`: las operaciones sobre la BD son asíncronas, y sequelize-cli espera a que termine cada una antes de continuar.
- `dropTable("Partidas")` en `down`: elimina la tabla, que es lo opuesto exacto de lo que hizo `up`.

La regla clave: `down` es el inverso de `up`

Si `up` hace...	`down` debe hacer...
`createTable`	`dropTable`

que son y como implementar services en js en el proyecto de jugo por turnos y acciones

Explicando qué son los servicios en JS para el juego.

Un service es un módulo que contiene la **lógica de negocio** (las reglas de tu juego) y que no sabe nada de HTTP. Recibe datos simples (ids, códigos), usa los modelos de Sequelize, y devuelve datos o lanza un error. El controlador lo llama y traduce el resultado a una respuesta.

Qué responsabilidad tiene cada capa

Capa	Hace	No debe hacer
Ruta	Asocia URL + método con un controlador	Lógica
Middleware	Cosas transversales: auth, validación, errores	Reglas del juego
Controlador	Lee `ctx` (params, body, usuario), llama al service, arma `ctx.body` y `ctx.status`	Consultar la BD o decidir reglas
Service	Reglas del juego, transacciones, consultas a modelos, lanzar errores	Usar `ctx`, `req`, `res` o status HTTP
Modelo	Estructura de datos, validaciones, a...ciones	Flujo del juego

2. Service: crear partida

javascript

```
// src/services/partidaService.js
const { sequelize, Partida, JugadorPartida } = require("../models");
const AppError = require("../errors/AppError");

const RECURSOS_INICIALES = 10;
```

```
function generarCodigo() {
  return Math.random().toString(36).substring(2, 8).toUpperCase();
}

async function crearPartida(usuarioId) {
  return sequelize.transaction(async (transaction) => {
    const partida = await Partida.create(
      { estado: "esperando", codigo: generarCodigo(), turnoActual: 1 },
      { transaction }
    );

    await JugadorPartida.create(
      {
        usuarioId,
        partidaId: partida.id,
        rol: "anfitrion",
        numeroJugador: 1,
        recursos: RECURSOS_INICIALES,
      },
      { transaction }
    );

    return partida;
```

```
async function unirsePorCodigo(usuarioId, codigo) {
  return sequelize.transaction(async (transaction) => {
    const partida = await Partida.findOne({
      where: { codigo },
      transaction,
      lock: transaction.LOCK.UPDATE,
    });

    if (!partida) {
      throw new AppError(404, "PARTIDA_NO_ENCONTRADA", "No existe una partida con ese
    }
    if (partida.estado !== "esperando") {
      throw new AppError(409, "PARTIDA_NO_DISPONIBLE", "La partida ya no acepta jugado
    }

    const yaEsta = await JugadorPartida.findOne({
      where: { partidaId: partida.id, usuarioId },
      transaction,
    });
    if (yaEsta) {
      throw new AppError(409, "YA_ESTAS_EN_LA_PARTIDA", "Ya perteneces a esta partida"
    }
```

```
    await JugadorPartida.create(
      {
        usuarioId,
        partidaId: partida.id,
        rol: "invitado",
        numeroJugador: 2,
        recursos: RECURSOS_INICIALES,
      },
      { transaction }
    );

    await partida.update({ estado: "en_juego" }, { transaction });
    return partida;
  });
}

module.exports = { crearPartida, unirsePorCodigo };
```

3. Service: terminar turno

javascript

```
// src/services/turnoService.js
const { sequelize, Partida, JugadorPartida } = require("../models");
const AppError = require("../errors/AppError");

async function terminarTurno(partidaId, usuarioId) {
  return sequelize.transaction(async (transaction) => {
    const partida = await Partida.findByPk(partidaId, {
      transaction,
      lock: transaction.LOCK.UPDATE,
    });

    if (!partida) {
      throw new AppError(404, "PARTIDA_NO_ENCONTRADA", "La partida no existe");
    }
    if (partida.estado !== "en_juego") {
      throw new AppError(409, "PARTIDA_NO_ACTIVA", "La partida no está en juego");
    }

    const jugador = await JugadorPartida.findOne({
      where: { partidaId, usuarioId },
      transaction,
    });
    if (!jugador) {
      throw new AppError(403, "NO_PERTENECES", "No participas en esta partida");
    }
    if (jugador.numeroJugador !== partida.turnoActual) {
      throw new AppError(403, "NO_ES_TU_TURNO", "No es tu turno");
    }

    const siguiente = partida.turnoActual === 1 ? 2 : 1;
    await partida.update({ turnoActual: siguiente }, { transaction });

    return partida;
  });
}

module.exports = { terminarTurno };
```

4. Controlador y ruta (delgados)

javascript

```
// src/controllers/partidaController.js
const partidaService = require("../services/partidaService");
const turnoService = require("../services/turnoService");

async function crear(ctx) {
  const partida = await partidaService.crearPartida(ctx.state.usuario.id);
  ctx.status = 201;
  ctx.body = partida;
}

async function unirse(ctx) {
  const { codigo } = ctx.request.body;
  const partida = await partidaService.unirsePorCodigo(ctx.state.usuario.id, codigo);
  ctx.body = partida;
}

async function terminarTurno(ctx) {
  const partida = await turnoService.terminarTurno(
    Number(ctx.params.id),
    ctx.state.usuario.id
  );
  ctx.body = partida;
```

```
module.exports = { crear, unirse, terminarTurno };
```

javascript

```
// src/routes/partidaRoutes.js
const Router = require("@koa/router");
const controller = require("../controllers/partidaController");
const { authRequired } = require("../middlewares/auth");

const router = new Router({ prefix: "/api/partidas" });

router.post("/", authRequired, controller.crear);
router.post("/unirse", authRequired, controller.unirse);
router.post("/:id/terminar-turno", authRequired, controller.terminarTurno);

module.exports = router;
```

- El controlador no tiene `try/catch`: si el service lanza un `AppError`, sube hasta el middleware global.
- `ctx.state.usuario`: lugar donde el middleware de auth deja al usuario autenticado.
- `Number(ctx.params.id)`: los params llegan como texto.
- Cada controlador hace tres cosas: extrae datos, llama al service, responde.

cmo escribo las funciones crear partida, unirse porcodigo, etc en el api service? y cómo armo el handler de los botones?

Construyendo el componente de menú con manejo de carga y navegación ▸

Se hace en dos capas: el **api service** solo habla HTTP (arma la petición y lanza un error si falla), y el **handler** del componente maneja el estado de la interfaz (`cargando` , `error`) y navega al lobby si todo sale bien.

1. La función base `request`

Como tu backend responde los errores como `{ codigo, mensaje }` , `request` debe leer ese cuerpo y convertirlo en un error de JavaScript. Así el componente solo hace `catch` y muestra el mensaje.

javascript

```
// src/services/api.js
const BASE_URL = import.meta.env.VITE_API_URL; // ej: http://localhost:8000/api

async function request(path, options = {}) {
  const token = localStorage.getItem("token");

  const headers = { "Content-Type": "application/json" };
  if (token) {
    headers.Authorization = "Bearer " + token;
  }
```

```
  const response = await fetch(BASE_URL + path, {
    method: options.method || "GET",
    headers: headers,
    body: options.body,
  });

  let data = null;
  try {
    data = await response.json();
  } catch (parseError) {
    data = null;
  }

  if (!response.ok) {
    const mensaje = data && data.mensaje ? data.mensaje : "No se pudo conectar con el
    const error = new Error(mensaje);
    error.codigo = data && data.codigo ? data.codigo : "ERROR_DESCONOCIDO";
    error.status = response.status;
    throw error;
  }

  return data;
}
```

- `localStorage.getItem("token")` : lee el JWT guardado en el login.
- `headers.Authorization = "Bearer " + to...` el formato que espera tu middleware `authRequired` . Solo se agrega si hay token.

- `fetch(BASE_URL + path, {...})` : hace la petición. Aquí no uso `...options` para que nadie pise el objeto `headers` por accidente.
- `try { await response.json() } catch` : si el servidor responde sin cuerpo o con algo que no es JSON (por ejemplo, se cayó), `json()` lanza error. Se captura para no ocultar el error real.
- `response.ok` : `fetch` no lanza error en 4xx/5xx, así que se revisa a mano.
- `error.codigo` y `error.status` : se agregan al error para que el componente pueda distinguir casos (por ejemplo, `PARTIDA_NO_ENCONTRADA`) si lo necesita.

2. Las funciones del api service

javascript

```
// src/services/api.js (continuación)
export function crearPartida() {
  return request("/partidas", { method: "POST" });
}

export function unirsePorCodigo(codigo) {
  return request("/partidas/unirse", {
    method: "POST",
    body: JSON.stringify({ codigo: codigo }),
  });
}
```

```
export function buscarRival() {
  return request("/partidas/buscar-rival", { method: "POST" });
}

export function obtenerPartida(partidaId) {
  return request("/partidas/" + partidaId, { method: "GET" });
}
```

- Cada función es una línea de conexión: método + ruta + body. No tiene lógica de interfaz.
- `JSON.stringify({ codigo: codigo })` : convierte el objeto a texto JSON, que es lo que viaja en el body. La clave `codigo` es la que lee tu controlador con `ctx.request.body` .
- `return request(...)` : devuelve la promesa; quien llame usa `await` .
- Las rutas coinciden con el prefijo `/api/partidas` del router (`/api` viene en `BASE_URL`).
- `buscarRival` y `obtenerPartida` necesitan sus endpoints en el backend (`POST /buscar-rival` y `GET /:id`). Los que hicimos antes fueron crear, unirse y terminar turno.

3. El handler en el menú

Como los tres botones comparten el mismo ciclo (activar `cargando` , llamar, navegar o mostrar error, desactivar `cargando`), conviene escribir ese ciclo **una sola vez.**

javascript

```
// src/pages/MenuPrincipal.jsx
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { crearPartida, unirsePorCodigo, buscarRival } from "../services/api";
```

```
function MenuPrincipal() {
  const navigate = useNavigate();
  const [cargando, setCargando] = useState(false);
  const [error, setError] = useState(null);
  const [codigo, setCodigo] = useState("");

  async function ejecutarAccion(accion) {
    if (cargando) {
      return;
    }

    setCargando(true);
    setError(null);

    try {
      const partida = await accion();
      navigate("/lobby/" + partida.id);
    } catch (err) {
      setError(err.message);
    } finally {
      setCargando(false);
    }
  }
```

```
function handleCrear() {
  ejecutarAccion(crearPartida);
}

function handleUnirse() {
  const codigoLimpio = codigo.trim().toUpperCase();

  if (codigoLimpio.length === 0) {
    setError("Ingresa el código de la partida");
    return;
  }

  ejecutarAccion(function () {
    return unirsePorCodigo(codigoLimpio);
  });
}

function handleBuscarRival() {
  ejecutarAccion(buscarRival);
}

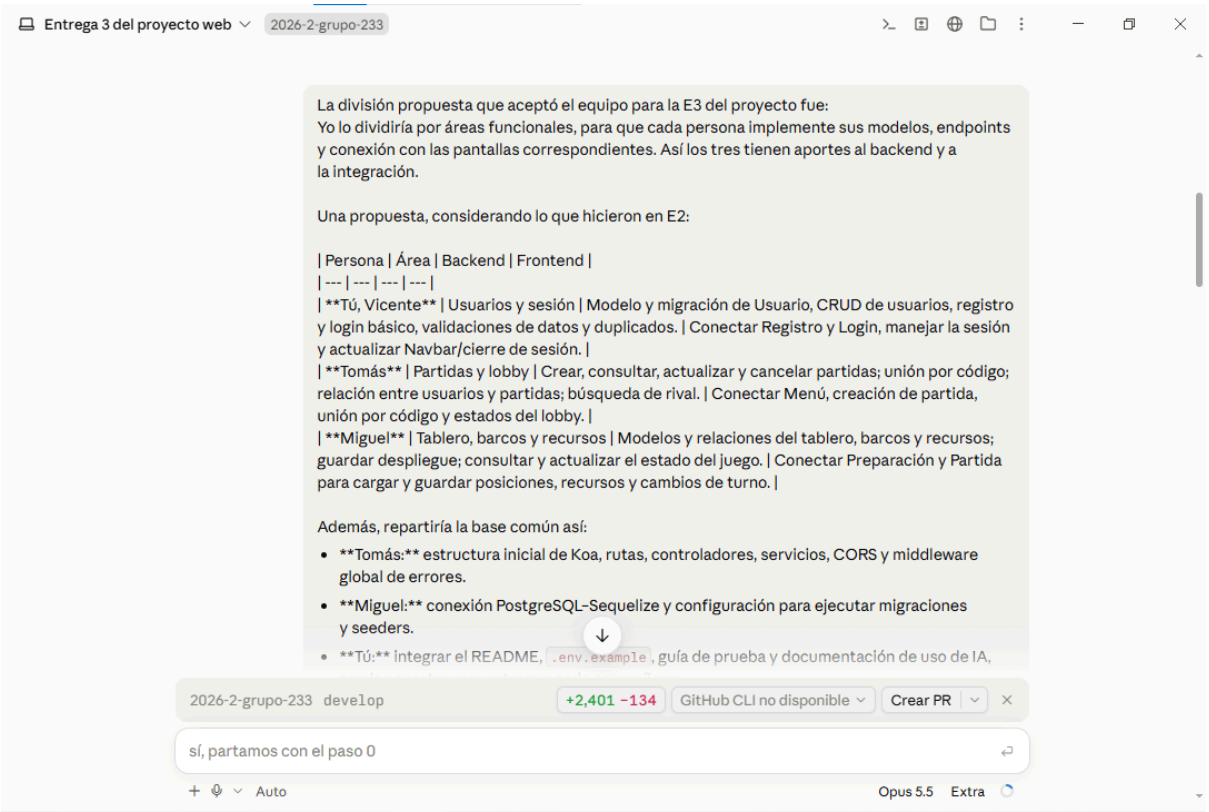
return (
  <div>
    <button onClick={handleCrear} disa ↓ ={cargando}>
      Crear partida
```

```
      <input
        value={codigo}
        onChange={function (event) {
          setCodigo(event.target.value);
        }}
        placeholder="Código de partida"
        disabled={cargando}
      />
      <button onClick={handleUnirse} disabled={cargando}>
        Unirse
      </button>

      <button onClick={handleBuscarRival} disabled={cargando}>
        {cargando ? "Buscando..." : "Buscar rival"}
      </button>

      {error && <p role="alert">{error}</p>}
    </div>
  );
}

export default MenuPrincipal;
```



- **Tú:** integrar el README, `.env.example`, guía de prueba y documentación de uso de IA, con los aportes que entregue cada compañero.

Cada uno debe completar estas cosas para su área:

- Modelos, migraciones, relaciones y validaciones.
- Endpoints y manejo de errores.
- Seeders con datos de prueba.
- Peticiones desde React.
- Pruebas de Postman y ejemplos de uso.
- Tareas en Kanban, commits y PR.

Así, la documentación y las pruebas se construyen durante el desarrollo.

El orden para trabajar sería:

1. **Acordar entre los tres el modelo final y el protocolo JSON.** Definir nombres de campos, rutas, respuestas y errores antes de programar.
2. **Levantar la estructura común y la conexión a PostgreSQL.** Con eso funcionando, cada uno desarrolla su área.
3. **Integrar un flujo pequeño cuanto antes:** registrar usuario → iniciar sesión → crear partida → unirse → desplegar barcos → consultar tablero.
4. **Completar el CRUD y probar juntos el ciclo básico de juego.**
5. **Cerrar documentación, colección de Postman y Kanban.**

La parte de Miguel probablemente [illegible] a más pesada, por la cantidad de entidades y relaciones. Para equilibrar, tú podrías implementar también el guardado del despliegue desde

La parte de Miguel probablemente sea la más pesada, por la cantidad de entidades y relaciones. Para equilibrar, tú podrías implementar también el guardado del despliegue desde Preparación, que ya era parte de tu trabajo en E2. Miguel definiría el endpoint y tú conectarías esa pantalla. Tomás podría asumir el endpoint de cambio de turno dentro de su área de Partidas.

Esta división les permite avanzar en paralelo, pero necesitan acordar desde el principio cómo se relacionan **Usuario, Partida y participación de cada jugador.** Es la unión entre las tres áreas.

Ahora, siendo que yo soy miguel dame un listado de en que orden hacer las tareas y como recomiendas partir, dime ademas que tareas ya están hechas y que tareas son dependientes entre si

Mostrar menos

hace 7 minutos

Antes de proponerte un orden reviso qué hay hecho en el backend, el frontend y el enunciado de la E3.

Ejecutado 5 comandos, leyó Entrega 3.pdf >

Me falta ver el diagrama ER de la E1 para saber qué tablas te tocan exactamente.

Ejecutado un comando, leyó Entrega 1 Diagramas_Mockups_Kanbann.pdf >

Ya tengo el panorama completo. Guardo el reparto de la E3 en memoria para las próximas sesiones.

Guardado 2 memorias, ejecutado un comando >

La entrega es el **domingo 18 de octubre a las 23:59**, así q… quedan unos 5 días. Revisé `develop` en los dos repos, el diagrama ER y de clases de la E1, el `protoco…_json.md` y el enunciado de la E3.

La entrega es el **domingo 18 de octubre a las 23:59**, así que te quedan unos 5 días. Revisé `develop` en los dos repos, el diagrama ER y de clases de la E1, el `protocolo_json.md` y el enunciado de la E3.

Qué ya está hecho

Backend (`develop`)

Cosa	Estado	Quién
Base de Koa: rutas, controladores, CORS, middleware de errores y `HttpError`	✅ Hecho	Tomás
Conexión Sequelize–PostgreSQL (config/database.js, models/index.js, `.sequelizerc`, scripts `db:create/migrate/seed/reset`, `/health`)	✅ Hecho	Tomás
Usuario: modelo, migración, CRUD, registro y login	✅ Hecho	Vicente
Seeders	❌ La carpeta está vacía	—
Partida y JugadorPartida	❌ No existen, y no hay ninguna rama pusheada	Tomás
Todo lo de tu área	❌	Tú

Ojo: la tarea de base común que te tocaba a ti (Sequelize, migraciones y seeders) **ya la hizo Tomás casi entera**. De esa parte solo te quedan los seeders.

Frontend (`develop`)

- ✅ Preparación ya arma el body exacto de `POST /games/:id/desplegar`: `armarDespliegue()` devuelve `[{tipo, casilla}]`, tal como pide el protocolo.
- ✅ Partida, Tablero, PanelRecursos y Bitácora ya consumen el objeto `ESTADO_PARTIDA` del protocolo (mocks/partida.js). Si tu `GET /games/:id` devuelve ese mismo formato, conectar Partida es casi solo cambiar el mock por un `fetch`.
- ✅ mocks/flota.js tiene las stats de cada barco, que pasan a ser constantes del servidor.
- ❌ Todavía no hay ningún `fetch` ni helper de API. Tampoco existe `VITE_API_URL`.
- ⚠️ La zona de despliegue está fija en las filas 1–4. El jugador 2 necesita las filas 7–10, y eso tiene que venir del servidor.

Tus tablas (de las 11 del modelo)

- **Tuyas:** `casilla`, `casilla_revelada`, `barco`, `catalogo_mejora`, `mejora_barco`, y los recursos de `jugador_partida`.
- **Sin dueño en la propuesta:** `accion`, `combate` y `evento_partida`. El criterio 1 pide "esquema completo", así que alguien las tiene que tomar. `accion` además alimenta la bitácora de jugadas que pide el enunciado.

Dependencias

```
Acuerdo de modelo y protocolo ⟶ bloquea todo
Partida + JugadorPartida (Tomás) ⟶ Casilla, Barco, CasillaRevelada y todos tus endpoints
Casilla ⟶ generarTablero() ⟶ Tomás lo llama al pasar la partida a DESPLIEGUE
Barco ⟶ POST desplegar ⟶ Vicente conecta Preparación
Barco + Casilla + CasillaRevelada + Usuario ⟶ GET /games/:id ⟶ conectas Partida
GET estado + turnoActual de Tomás ⟶ acción MOVER ⟶ mover en Partida
Terminar turno (Tomás) ⟶ reinicia accionesRestantes, que son campos tuyos
Seeder usuarios (Vicente) ⟶ seeder partidas (Tomás) ⟶ tu seeder de tablero y barcos
Todos los endpoints ⟶ Postman + sección del README
```

Lo que no depende de nadie y puedes partir hoy:

- Catálogo de mejoras: modelo, migración y seeder (sin FK).
- Constantes de reglas: stats de barcos, costos y zonas.
- Funciones puras para generar el tablero, calcular distancias y la visibilidad.

Decisiones que hay que cerrar con el equipo antes de programar

Al lado de cada una va mi recomendación:

1. **¿Quién crea `JugadorPartida` y con qué columnas?** Recomiendo que la haga Tomás, pero con todas las columnas acordadas desde el inicio, incluidos `combustible`, `municion`, `chatarra`, `accionesRestantes` y `listo`.
2. **¿Cómo se identifica al jugador sin token?** El protocolo usa `Bearer token`, pero el JWT queda para la E4. Recomiendo un header temporal `x-usuario-id`, documentado en el README como ajuste de la E3.
3. **¿IDs numéricos o `"p-1"` / `"b-1"`?** Recomiendo números de la base y documentar el cambio.
4. **¿`jugadorId` es el id del usuario o el número de jugador 1/2?** El protocolo usa los dos sentidos. Recomiendo `jugadorId = Usuario.id` y agregar `numeroJugador` en `yo` para la zona de despliegue.
5. **¿Quién es dueño de `GET /games/:id`?** Es un solo endpoint para el lobby (Tomás) y para el juego (tú). Recomiendo que tú armes el estado filtrado y Tomás devuelva la versión reducida mientras la partida esté en lobby.
6. **Orden de migraciones.** Usan prefijo numérico (`001-...`), así que Partida y JugadorPartida van antes que las tuyas.

7. **Quién toma** `accion`, `combate` y `evento_partida`.
8. **Ruta del front:** `/tablero/:partidaId` en vez de pasar el estado por `location.state`.

Orden recomendado

#	Tarea	¿Bloqueada?	Día sugerido
0	Mandar las decisiones al grupo y levantar el backend en local (Postgres, `.env`, `db:migrate`, `/health`)	No	Mar 13
1	Seeders funcionando, más el modelo, migración y seed del catálogo de mejoras	No	Mié 14
2	Constantes de reglas y funciones puras (generar tablero, distancia, visibilidad)	No	Mié 14
3	Modelos y migraciones de Casilla, Barco, CasillaRevelada y MejoraBarco, con asociaciones y validaciones	Necesita el esquema de Partida y JugadorPartida	Jue 15
4	Servicio `generarTablero(partidaId)` y avisarle a Tomás	Necesita el paso 3	Jue 15
5	`POST /games/:id/desplegar` (+ `DELET` ↓ a "Limpiar") y avisarle a Vicente	Necesita el paso 3	Jue 15
6	`GET /games/:id` con el estado filtrado por jugador, más lecturas de tablero y barcos	Necesita los pasos 3 y 5	Vie 16
7	Acción `MOVER`: actualiza posición, combustible y acciones (cubre el "U" del CRUD)	Necesita el paso 6 y el turno de Tomás	Vie 16
8	Seeder de una partida en curso con casillas y barcos	Necesita los seeders de Vicente y Tomás	Sáb 17
9	Frontend Partida: cargar desde `GET`, mover con `POST` y refrescar	Necesita los pasos 6 y 7	Sáb 17
10	Requests de Postman, tu sección del README y el Kanban	Necesita todo lo anterior	Dom 18

Cómo partir

Hoy mandaría al grupo las decisiones 1, 2, 4 y 5. Son las que, si salen mal, nos hacen rehacer FKs o endpoints. Mientras responden, levantamos tu backend local y hacemos los pasos 1 y 2, que no dependen de nadie y te dejan commits propios desde el primer día.

Una cosa importante: como estamos usando IA, la E3 exige declarar los prompts y adjuntar capturas. Si no se declara bien, el descuento puede llegar hasta 100 puntos. Te conviene ir guardando los prompts de esta sesión.

¿Partimos con el paso 0?

como creas el catalogo de mejoras

El catálogo se arma con tres archivos. Cada uno cumple un rol distinto y Sequelize los conecta entre sí.

1. La migración: crea la tabla en PostgreSQL

src/migrations/004-create-catalogo-mejoras.js es la instrucción para construir la tabla real en la base de datos:

```
async up(queryInterface, Sequelize) {
  await queryInterface.createTable('CatalogoMejoras', {
    id: { type: Sequelize.INTEGER, autoIncrement: true, primaryKey: true, ...
},
    nombre: { type: Sequelize.STRING, allowNull: false, unique: true },
    ...
```

- `up` crea la tabla con sus columnas. Al nivel de la base solo exige lo básico: `NOT NULL` y que `nombre` sea `UNIQUE`.
- `down` hace lo contrario, `dropTable`, por si hay que deshacer la migración.

Cuando corres `npm run db:migrate`, `sequelize-cli` revisa la carpeta `src/migrations` (la ruta está en `.sequelizerc`), ejecuta en orden alfabético las migraciones que faltan y anota cada una en la tabla `SequelizeMeta`. Por eso el prefijo `004` importa: decide el orden en que corren.

2. El modelo: cómo el backend usa esa tabla desde JavaScript

src/models/catalogoMejora.js le describe la tabla a Sequelize y le agrega las reglas del juego como validaciones:

```
nombre: {
  ...
  validate: {
    isIn: {
      args: [['BLINDAJE', 'MOTORES', 'ARTILLERIA', 'RADAR']],
      msg: 'La mejora debe ser BLINDAJE, MOTORES, ARTILLERIA o RADAR.'}}},
```

- Las validaciones de `isIn` y `min` viven acá y no en la migración. Se revisan **antes** de escribir en la base, cada vez que el código hace `create` o `update`. Esto es lo que pide el enunciado con "validaciones a nivel de ORM".
- No hubo que registrarlo en ningún lado. El src/models/index.js que dejó Tomás lee todos los archivos de la carpeta y los carga solo. Por eso después se puede hacer `const { CatalogoMejora } = require('../models')`.

3. El seeder: llena la tabla con los datos iniciales

src/seeders/004-catalogo-mejoras.js inserta las 4 mejoras de la regla 30:

```
const MEJORAS = [
  { nombre: 'BLINDAJE', atributo: 'casco', costoChatarra: 3 },
  ...];

await queryInterface.bulkInsert('CatalogoMejoras',
  MEJORAS.map((mejora) => ({ ...mejora, incremento: 1, createdAt: ahora,
updatedAt: ahora })));
```

- `up` inserta las 4 filas de una vez. `creat` y `updatedAt` se ponen a mano porque `bulkInsert` escribe directo en la base, sin pasar por el modelo. Por lo mismo, el **seeder no pasa por las validaciones del modelo**: los datos tienen que estar bien escritos en el archivo.
- `down` borra solo esas 4 mejoras, buscándolas por nombre.

Con `npm run db:seed` corren los seeders. Gracias a la línea `seederStorage: 'sequelize'` que agregué en `database.js`, cada seeder queda anotado en la tabla `SequelizeData` y no se vuelve a ejecutar. Sin esa línea, cada `db:seed` intentaría insertar las mejoras otra vez y fallaría por el `unique` de `nombre`.

Resumen del flujo

```
npm run db:migrate  ->  migración 004  ->  crea la tabla vacía       (anota
en SequelizeMeta)
npm run db:seed     ->  seeder 004     ->  inserta las 4 mejoras     (anota
en SequelizeData)
el backend          ->  modelo         ->  lee y escribe validando
(require('../models'))
```

En resumen: la migración define **dónde** se guardan los datos, el modelo define **qué** datos son válidos y el seeder pone **los datos con los que parte** el juego. Más adelante, la acción "Mejorar" va a leer de esta tabla el costo y el stat de cada mejora, en vez de tener esos valores escritos en el código.

abri el pr y mis compañeros dicen q hay conflicto, nose porque será, creo q es porque trabajamos en una rama desactualizada

Reviso qué cambió en `develop` desde que creamos la rama.

Ejecutado 2 comandos ›

Encontré el conflicto: tú y Tomás agregaron la misma línea en src/config/database.js, pero tú con un comentario y él sin él en su commit ya mergeado en `develop`. Lo resolveré trayendo `develop` a tu rama y quedándome con la versión de Tomás, ya que la línea es idéntica.

Merged develop and resolved the config conflict ›

El conflicto quedó resuelto y el PR ahora solo muestra los 3 archivos del catálogo. Antes de subirlo, voy a probar todo junto desde una base vacía: migraciones 002, 003 y 004, más los seeders de ambos.

Reset database with all migrations and seeders, then checked counts ›

Todo funciona junto:

- **Migraciones:** corren en orden, 001 → 002 (partidas) → 003 (jugadores de partida) → 004 (tu catálogo).
- **Seeders:** el de Tomás (`002-demo-partidas`) y el tuyo (`004-catalogo-mejoras`) corren sin errores.
- **Base:** quedó con 9 usuarios, 4 partidas, 7 jugadores de partida y 4 mejoras.
- **Servidor:** `/health` responde OK. `GET /games` responde 401 sin el header `x-usuario-id`, que es lo esperado con el middleware nuevo de Tomás.

Qué cambió en tu rama

- Hice un commit de merge, `f729169`, a tu nombre y sin coautor. Trae todo lo nuevo de `develop`.
- En `database.js` quedó la versión de Tomás, con una sola línea `seederStorage`. Tu commit `6b00896` sigue en el historial, pero ya no cambia nada respecto de `develop`.
- El PR ahora muestra **solo tus 3 archivos**: modelo, migración y seeder del catálogo.

Conviene corregir una línea de la descripción del PR. El punto del `seederStorage` ya no aplica, porque Tomás agregó la misma línea en su PR. Puedes borrar ese punto o cambiarlo por algo como: "La configuración de `seederStorage` ya venía en el PR de partidas de Tomás".

¿Qué hacen up y down en una migración?

`up` aplica el cambio en la base y `down` lo deshace. Son dos funciones dentro del mismo archivo, una para cada dirección.

Con tu migración 006 como ejemplo

```
async up(queryInterface, Sequelize) {
  await queryInterface.createTable('Barcos', { ... });       // crea la tabla
  await queryInterface.addConstraint('Barcos', { ... });     // le agrega el unique
(jugadorPartidaId, tipo)
},

async down(queryInterface) {
  await queryInterface.dropTable('Barcos');                  // borra la tabla, y con ella su
unique
},
```

- `up`: lleva la base **hacia adelante**. Antes no existía la tabla `Barcos` y después sí.
- `down`: la lleva **hacia atrás**, al estado exacto en que estaba antes del `up`.

Cuándo se ejecuta cada una

Comando	Qué corre
`npm run db:migrate`	El `up` de cada migración pendiente, en orden: 005 → 006 → 007 → 008
`npx sequelize-cli db:migrate:undo`	El `down` de la última migración ejecutada, solo una

`npm run db:reset`	Ningún `down`: borra la base entera y después corre todos los `up`

En el paso 3 probé las dos direcciones: corrí `db:migrate:undo` 4 veces y se ejecutaron los `down` en orden inverso, 008 → 007 → 006 → 005. Después volví a migrar y todo subió de nuevo sin errores.

Por qué el orden inverso importa

`CasillasReveladas` tiene una FK que apunta a `Casillas`. Si se intentara borrar `Casillas` primero, PostgreSQL lo rechazaría, porque otra tabla todavía depende de ella. Por eso los `down` corren al revés: primero se borra la tabla que depende (007) y después la tabla de la que depende (005).

La regla de oro

El `down` tiene que deshacer exactamente lo que hizo el `up`, ni más ni menos. En las tuyas basta con `dropTable`, porque el `up` solo creó una tabla y sus restricciones, y esas restricciones se borran junto con la tabla.

La migración 003 de Tomás es un ejemplo más delicado. Su `up`, además de crear `JugadoresPartida`, le agrega dos FK a **otra** tabla, `Partidas` (`turnoActualId` y `ganadorId`). Por eso su `down` primero tiene que sacar esas dos FK de `Partidas` y recién después borrar `JugadoresPartida`. Si solo hiciera `dropTable`, quedaría mal deshecho.

Para qué sirve en la práctica

- **Corregir un error mientras desarrollas:** fue lo que hicimos con la 008. Deshice la migración, cambié `RESTRICT` por `NO ACTION` y la volví a correr, sin borrar la base completa.
- **Volver atrás en producción (Render):** si una migración nueva rompe algo, se deshace con su `down` sin perder los datos de las demás tablas, cosa que `db:reset` no permite.

hace 6 minutos

¿Qué revisa exactamente /health?

Read the health route and controller >

`/health` revisa dos cosas: **que el servidor esté vivo** y **que pueda conectarse a la base de datos**. Nada más.

El código (lo escribió Tomás)

```
// src/routes/health.routes.js
router.get('/health', getHealth);

// src/controllers/health.controller.js
async function getHealth(ctx) {
  await sequelize.authenticate();
  ctx.body = { estado: 'OK', baseDeDatos: 'conectada' };
}
```

Paso a paso

1. **Llega la petición `GET /health`.** Si el servidor responde algo, ya sabes que Koa está corriendo, que el puerto 3000 está escuchando y que las rutas están cargadas.
2. **`sequelize.authenticate()`** abre una conexión a PostgreSQL con los datos del `.env` (`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD` y `DB_NAME`) y manda una consulta mínima. Esa consulta es el `SELECT 1+1 AS result` que viste en la consola del servidor. No lee ninguna tabla: solo comprueba que PostgreSQL contesta.
3. **Si funciona**, responde `200` con `{"estado":"OK","baseDeDatos":"conectada"}`.

4. **Si falla,** `authenticate()` lanza un error. El middleware de errores lo atrapa y responde `500` con `ERROR_INTERNO`. Pasaría, por ejemplo, si la clave del `.env` está mala, si Ubuntu está cerrado o si la base no existe.

Lo que sí te dice y lo que no

Revisa	No revisa
Que el servidor Koa está corriendo	Que las migraciones se hayan ejecutado
Que el `.env` tiene credenciales válidas	Que existan las tablas `Barcos`, `Partidas`, etc.
Que PostgreSQL está prendido y acepta la conexión	Que haya datos de los seeders
Que la base `guerra_pacifico_dev` existe	Que los demás endpoints funcionen bien

Por ejemplo, con una base recién creada y sin migrar, `/health` igual responde OK, aunque `GET /games` fallaría porque la tabla `Partidas` todavía no existe.

Para qué lo usamos

- **Al levantar el entorno (paso 0):** fue la prueba de que tu `.env`, el PostgreSQL de Ubuntu y Node estaban bien conectados.
- **Para diagnosticar:** si un endpoint falla, `/health` te dice rápido si el problema es la conexión a la base o el código del endpoint.
- **En Render:** sirve para revisar que el servidor desplegado esté vivo y conectado a la base de la nube. Render incluso permite configurar una ruta de *health check* para vigilar el servicio.

¿Por qué las migraciones llevan número (001, 004...) y qué pasa si alguien renombra o edita una que ya se corrió?

Las dos preguntas tienen la misma raíz: **Sequelize identifica cada migración por el nombre de su archivo.**

Por qué llevan número

1. El número define el orden. `sequelize-cli` ejecuta las migraciones pendientes en orden alfabético por nombre de archivo, y el número al inicio es lo que hace que ese orden sea el correcto:

```
001-create-usuarios.js
002-create-partidas.js
003-create-jugadores-partida.js      -> tiene FK a Usuarios y Partidas
004-create-catalogo-mejoras.js
005-create-casillas.js               -> tiene FK a Partidas
006-create-barcos.js                 -> tiene FK a JugadoresPartida
007-create-casillas-reveladas.js     -> tiene FK a JugadoresPartida y Casillas
008-create-mejoras-barco.js          -> tiene FK a Barcos y CatalogoMejoras
```

Una FK solo se puede crear si la tabla a la que apunta ya existe. Si `006-create-barcos` corriera antes que `003-create-jugadores-partida`, PostgreSQL daría error porque `JugadoresPartida` todavía no existiría.

Cómo sabe Sequelize cuáles ya corrieron

Cada vez que corre el `up` de una migración, Sequelize anota **el nombre del archivo** en la tabla `SequelizeMeta`:

```
 name
-------------------------------
 001-create-usuarios.js
 002-create-partidas.js
 ...
 008-create-mejoras-barco.js
```

Cuando corres `db:migrate`, compara los archivos de la carpeta con esa lista y ejecuta solo los que no aparecen. **No revisa el contenido de los archivos, solo los nombres.**

Qué pasa si alguien renombra una que ya corrió

Sequelize ve un nombre que no está en `SequelizeMeta`, así que cree que es una migración nueva y vuelve a ejecutar su `up`:

```
002-create-partidas.js  ->  createTable('Partidas')  ->  ERROR: relation "Partidas" already
exists
```

Además, el nombre viejo queda anotado en `SequelizeMeta` aunque su archivo ya no exista. Eso fue lo que pasó con el commit de Tomás, y por eso a tu compañero le recomendaron `db:reset`.

Qué pasa si alguien edita una que ya corrió

Esto es más peligroso, porque no aparece ningún error. El nombre sigue en `SequelizeMeta`, así que Sequelize nunca la vuelve a ejecutar y el cambio simplemente no llega a tu base.

Por ejemplo: Tomás agregó `numeroJugador` dentro de la migración 003, que ya existía. A quien ya había corrido la 003 vieja, `db:migrate` no le agrega esa columna. Entonces:

- En el computador de Tomás, con la base nueva, todo funciona.
- En el de otro compañero, con la base vieja, el backend falla con *"column numeroJugador does not exist"*.
- En Render sería peor, porque ahí no se puede hacer `db:reset` sin borrar los datos de producción.

Cómo se hace bien

Situación	Qué hacer
La migración **todavía no está en** `develop` (solo en tu rama)	Se puede editar: `db:migrate:undo`, editas y `db:migrate`. Es lo que hicimos con la 008 al cambiar `RESTRICT` por `NO ACTION`.
La migración **ya está en** `develop` o en Render	No se toca. Se crea una nueva, por ejemplo `009-add-numero-jugador.js`, con `addColumn` en el `up` y `removeColumn` en el `down`. Así a todos les basta con `db:migrate`.

En resumen: el número decide **en qué orden** corren las migraciones, y el nombre completo es **cómo Sequelize recuerda** cuáles ya ejecutó. Si cambias el nombre, la vuelve a correr. Si cambias el contenido, no se entera.

¿Por qué el seeder no pasa por las validaciones del modelo?

Porque el seeder no usa el modelo. Escribe directo en la tabla, por debajo de la capa donde viven las validaciones.

Las dos capas

```
tu código del backend  ->  Modelo (CatalogoMejora)  ->  valida isIn, min, etc.  ->  INSERT en
PostgreSQL
el seeder              ->  queryInterface.bulkInsert  ---------------------------->  INSERT en
PostgreSQL
```

En tu seeder:

```
async up(queryInterface) {
  await queryInterface.bulkInsert('CatalogoMejoras', [...]);
}
```

`queryInterface` es la misma herramienta de bajo nivel que usan las migraciones (`createTable`, `addConstraint`). `bulkInsert` no conoce el modelo `CatalogoMejora`: recibe el **nombre de la tabla** y arma el `INSERT` en SQL directamente. Por eso no pasa por nada de lo que definiste en `catalogoMejora.js`.

Ejemplo concreto: si el seeder tuviera `{ nombre: 'LASER', costoChatarra: 0 }`, **se insertaría igual**, porque PostgreSQL no sabe que LASER no es una mejora válida. En cambio, si el seeder intentara insertar dos veces `RADAR`, fallaría, porque el `unique` sí está en la tabla. Es justo lo que pasaba antes de configurar `seederStorage`.

Por qué Sequelize lo hace así a propósito

Las migraciones y los seeders son **historia**, y tienen que poder correr igual dentro de meses. Los modelos, en cambio, van cambiando.

Supón que en la E4 alguien renombra `costoChatarra` a `costo` en el modelo y crea una migración nueva para cambiar la columna. Si tu seeder 004 usara el modelo, intentaría escribir `costo` en una tabla que en ese punto de la historia todavía tiene `costoChatarra`, y `db:reset` se rompería. Con `queryInterface`, el seeder 004 trabaja contra la tabla tal como existe después de la migración 004, sin importar cómo esté el modelo hoy.

Qué significa para ti

- **La responsabilidad pasa a ti:** los datos del seeder tienen que estar bien escritos. En el catálogo los copié de la regla 30 y lo comprobamos consultando la tabla.
- **Para el seeder de partidas con barcos y casillas (paso 8):** hay dos caminos.
  - Usar `queryInterface.bulkInsert`, que es lo estándar, revisando bien los datos.
  - Usar el modelo con `Barco.bulkCreate(datos, { validate: true })`. Así se aplican las validaciones, a cambio de que el seeder quede atado a cómo esté el modelo.

Esa decisión la vemos cuando lleguemos al paso 8.

hace 3 minutos

¿Por qué hay validaciones en el modelo (isIn, min) y además restricciones en la migración (unique, allowNull)? ¿Qué protege cada una?

Porque protegen contra cosas distintas. Las validaciones del modelo **detectan los errores temprano y los explican bien**. Las restricciones de la base **garantizan la integridad pase lo que pase**. En tu código ya existen las dos capas, y más adelante se suma una tercera: los servicios.

Restricciones de la migración: la última línea de defensa

Viven **en la tabla misma**, dentro de PostgreSQL. Se cumplen siempre, venga de donde venga el dato:

- tu backend
- un seeder (que, como vimos, se salta el modelo)
- alguien que escribe SQL directo en `psql`
- código de Tomás o Vicente que no use tu modelo

Ejemplos en tu código:

- `allowNull: false` en `x` e `y`: ninguna casilla queda sin coordenada.
- `unique (partidaId, x, y)`: una partida nunca tiene dos casillas en el mismo lugar.
- La FK `jugadorPartidaId` con `CASCADE`: no hay barcos de jugadores que no existen, y si se borra la partida no quedan barcos huérfanos.

Lo que solo la base puede garantizar es la unicidad. Imagina que el jugador hace doble clic en "comprar Blindaje" y llegan dos peticiones al mismo tiempo. Las dos revisan "¿este barco ya tiene Blindaje?", las dos ven que no, y las dos intentan insertar. El código JavaScript no puede evitarlo, porque las dos revisiones ocurrieron antes de cualquier inserción. El `unique (bar…, catalogoMejoraId)` de la tabla sí lo evita: la segunda inserción falla.

Validaciones del modelo: la primera línea

Viven en **JavaScript**, en tu archivo del modelo, y se revisan **antes** de mandar la consulta a la base:

- **Mensajes claros en español.** Sin validación, PostgreSQL respondería algo críptico. Con ella, el usuario ve *"La coordenada x del barco debe ser un entero entre 1 y 10."*
- **Reglas que en la tabla serían incómodas o imposibles.** Tu validador `hundidoSinCasco` mira dos campos a la vez (`aFlote` y `cascoActual`), e `isIn` revisa contra la lista de `TIPOS_BARCO` de `reglas.js`.
- **Todos los errores de una vez.** En la prueba, un barco con datos malos devolvió 4 mensajes juntos, y no solo el primero.

Cómo se ven para quien usa la API

El middleware de errores de Tomás traduce cada tipo de error a una respuesta HTTP:

Origen	Error de Sequelize	Respuesta
Validación del modelo (`isIn`, `min`, `hundidoSinCasco`)	`ValidationError`	**400** `VALIDACION_FALLIDA` con tus mensajes
`allowNull` (está en los dos lados, el modelo lo detecta primero)	`ValidationError`	**400**
`unique` de la tabla	`UniqueConstraintError`	**409** `REGISTRO_DUPLICADO`

Hay un detalle con `unique`: también lo puse en el modelo (`indexes`), pero ahí no revisa nada. Sequelize no consulta la base antes de insertar para ver si hay repetidos. Solo lo usa para entender el error cuando la tabla rechaza el duplicado. La protección real está en la migración.

La tercera capa: los servicios

Hay reglas que no pueden ir en ninguna de las dos, porque **dependen del contexto de la partida** y no de un registro por sí solo:

- "Un barco tiene máximo 2 mejoras": para saberlo hay que contar las demás filas.
- "Es tu turno", "te quedan acciones", "tienes chatarra suficiente".

Esas reglas van en los servicios que vienen en los próximos pasos, con los códigos del protocolo (`LIMITE_MEJORAS`, `NO_ES_TU_TURNO`, etc.).

En resumen

Capa	Pregunta que responde	Dónde está
Servicio	¿Esta acción está permitida en este momento del juego?	`services/` (próximos pasos)
Modelo	¿Este registro, por sí solo, tiene datos válidos?	models/barco.js, etc.
Base de datos	¿La información guardada es consistente, aunque falle todo lo demás?	`migrations/`

Cada capa ataja lo que la anterior no puede. Si una falla o alguien se la salta, la siguiente la cubre.

¿Cómo está funcionando Claude en esta sesión? Mal Regular Bueno ×

¿Qué diferencia hay entre hasMany, belongsTo y belongsToMany? ¿Dónde usé cada una?

Las tres describen relaciones entre tablas. La forma más fácil de distinguirlas es preguntar **en qué tabla está la FK**.

La regla para distinguirlas

Asociación	Significado	¿Dónde está la FK?
`A.belongsTo(B)`	"A pertenece a un B"	En la tabla de **A** (A tiene la columna `bId`)
`B.hasMany(A)`	"B tiene muchos A"	En la tabla de **A** (la misma FK, vista desde el otro lado)
`A.belongsToMany(B, { through: T })`	"Muchos A con muchos B"	En una **tabla intermedia T**, que tiene las dos FK

`belongsTo` y `hasMany` son **la misma relación 1:N vista desde cada lado**. Se declaran en pareja para poder navegarla en las dos direcciones.

Dónde usé cada una

1:N con `belongsTo` + `hasMany`:

```
// casilla.js: la tabla Casillas tiene la columna partidaId
Casilla.belongsTo(models.Partida, { foreignKey: '...idaId', as: 'partida' });
models.Partida.hasMany(Casilla, { foreignKey: 'pa...aId', as: 'casillas' });   // 1 partida ->
models.JugadorPartida.hasMany(Barco, { foreignKey: 'jugadorPartidaId', as: 'barcos' });   // 1
jugador -> 4 barcos
```

N:M con `belongsToMany`:

```
// barco.js: un barco compra varias mejoras y cada mejora la compran muchos barcos
Barco.belongsToMany(models.CatalogoMejora, { through: models.MejoraBarco, ..., as: 'mejoras' });

// casillaRevelada.js: un jugador descubre muchas casillas y una casilla la pueden descubrir los
dos jugadores
models.JugadorPartida.belongsToMany(models.Casilla, { through: CasillaRevelada, ..., as:
'casillasReveladas' });
```

`belongsTo` desde las tablas intermedias: `MejoraBarco` y `CasillaRevelada` tienen dos FK cada una, así que cada una declara dos `belongsTo`. Por ejemplo, `MejoraBarco.belongsTo(Barco)` y `MejoraBarco.belongsTo(CatalogoMejora)`.

Tomás usa el mismo patrón N:M: `Partida.belongsToMany(Usuario, { through: JugadorPartida })`.

Por qué las tablas intermedias son modelos propios

En un N:M simple, la tabla intermedia solo tendría las dos FK. Las tuyas necesitan **datos extra sobre la relación**:

- `CasillasReveladas` guarda `rondaDescubierta`, la ronda en que el jugador descubrió esa casilla.
- `MejorasBarco` guarda `rondaAdquirida`, la ronda en que el barco compró esa mejora.

Por eso son modelos con sus propias validaciones, y no tablas ocultas. Ese dato extra se puede leer desde la relación: en la prueba, `jugadorPartida.casillasRevel...0].CasillaRevelada.rondaDescubierta` devolvió `0`.

Qué ganas al declararlas

Las asociaciones no crean columnas ni FK en la base. Eso lo hace la migración, con `references`. Lo que hacen las asociaciones es enseñarle a Sequelize a hacer los `JOIN`. A cambio, puedes:

```
// traer una partida con sus 100 casillas en una sola consulta
Partida.findByPk(id, { include: [{ association: 'casillas' }] });

// métodos que Sequelize crea solo
barco.getJugadorPartida();          // por el belongsTo
jugadorPartida.getBarcos();         // por el hasMany
barco.getMejoras();                 // por el belongsToMany
barco.addMejora(blindaje, { through: { rondaAdquirida: 3 } });   // inserta en MejorasBarco
```

Los nombres que pusimos en `as` (`'casillas'`, `'barcos'`, `'mejoras'`) son los que usas en esos `include` y en los métodos. Eso es justo lo que va a necesitar el paso 6 para armar el estado de la partida: una consulta que traiga partida → jugadores → barcos → mejoras, y además las casillas.

¿Por qué el servidor nunca le manda el estado completo de la partida al cliente?

Porque Guerra del Pacífico es un juego de **información oculta**: la niebla de guerra es el centro del juego. **Todo lo que llega al navegador, el jugador lo puede ver**, aunque la pantalla no lo muestre.

El problema de esconderlo en el frontend

Supón que `GET /games/:id` devolviera la flota enemiga completa con un campo `visible: false`, y que React simplemente no dibujara esos barcos:

```
"flotaRival": [
  { "tipo": "ACORAZADO", "casilla": { "x": 4, "y": 9 }, "casco": 4, "visible": false }
]
```

El tablero se vería con niebla, pero bastaría con apretar **F12** → pestaña **Network** → clic en la petición para ver dónde está cada barco enemigo. No hace falta saber programar, y el juego deja de tener sentido. Esto lo dice la regla 22 de tu Propuesta, y también aparece como "no negociable" en el `CLAUDE.md` del equipo.

La única forma segura es que ese dato no exista en la respuesta. No basta con marcarlo como oculto ni con no dibujarlo: el servidor directamente no lo incluye.

Qué filtra el servidor (protocolo, punto 3.2)

Dato	Lo que recibe cada jugador
Su flota y sus recursos	Todo, siempre
Recursos del rival	**Nunca.** Solo cuántos barcos le quedan a flote (`barcosAFlote`)
Casillas del mapa	Solo las que están dentro del radio de detección de sus barcos, más las que ya reveló antes (`CasillasReveladas`)
Corrientes peligrosas	Solo las que ya sufrió o descubrió con el Vigía
Barcos enemigos	Solo los que están dentro de su [illegible]. Si salen, queda la última posición conocida como "contacto perdido"

Casco del enemigo	El valor real si está en su radio. Si no, `null`, incluso cuando le acaba de pegar un disparo (regla 24)

Por eso el servidor arma **dos objetos distintos**, uno por jugador, a partir de la misma partida en la base.

La otra mitad: el cliente no decide nada

El mismo principio funciona al revés. El cliente solo manda **intenciones** ("mueve este barco a esta casilla"), y el servidor valida y calcula el resultado. Si el navegador calculara cosas como el dado del combate, el daño o el combustible gastado, el jugador podría modificar el JavaScript y enviar *"gané el combate"*. Por eso toda la aleatoriedad también sale del servidor (regla 34), y por eso `generarCasillas()` usa `crypto` en el backend.

En resumen: el servidor es **la única fuente de verdad**. Guarda el estado completo en la base, pero a cada jugador le muestra solo lo que le corresponde ver.

Cómo lo vas a implementar (paso 6)

Ahí se juntan las piezas que ya hiciste:

1. Se cargan de la base la partida, los dos jugadores, sus barcos y las casillas, gracias a las asociaciones del paso 3.
2. Para el jugador que pregunta (el que manda `x-usuario-id`), se calculan las casillas visibles con `casillasEnRadio(barco, barco.deteccion)` para cada uno de sus barcos a flote, más sus `CasillasReveladas`.
3. Se incluyen solo los barcos enemigos que caen en esas casillas.
4. Se arma el JSON del protocolo con esa vista filtrada.

¿Por qué el tablero se genera en el servidor y no en el frontend?

Por cuatro razones. Las dos primeras son las más importantes.

1. El mapa tiene información oculta. Las corrientes peligrosas y las casillas que todavía no descubres no se pueden ver (reglas 10 y 19). Si el navegador generara el tablero, conocería el mapa completo: bastaría con abrir F12 para ver dónde están todas las corrientes y los pozos. Como lo genera el servidor, el mapa completo nunca sale de la base de datos, y a cada jugador se le manda solo lo que puede ver.

2. El azar no se puede manipular (regla 34). Si el cliente sorteara el mapa, un jugador podría modificar el JavaScript para que los pozos petroleros caigan pegados a su zona, o regenerar el mapa hasta que le salga uno favorable. En el servidor nadie puede tocar ese sorteo. Además, `generarCasillas()` usa `crypto` en vez de `Math.random`, así que el resultado tampoco se puede predecir.

3. Los dos jugadores tienen que jugar en el mismo mapa. Si cada navegador generara su propio tablero, cada jugador vería pozos y corrientes en lugares distintos y la partida no tendría sentido. El servidor genera uno solo y lo guarda en la base, y esa es la única versión que vale para los dos. Es la idea del servidor como única fuente de verdad.

4. El tablero tiene que sobrevivir a una recarga. El enunciado exige que todo el estado persistente esté en PostgreSQL. Como las 100 casillas quedan guardadas en la tabla `Casillas`, si un jugador recarga la página o vuelve más tarde, el mapa sigue siendo el mismo.

En resumen: el frontend solo **muestra** el tablero, con la parte que cada jugador tiene permitido ver. El servidor es el que lo **crea**, lo **guarda** y decide qué mostrar.

¿Cómo sabe el servidor cuál es tu zona de despliegue?

El cliente nunca le dice al servidor cuál es su zona. El servidor la deduce siguiendo una cadena de datos que ya están guardados:

1. Quién eres: viene del header `X-Usuario-Id`. El middleware `usuarioActual` de Tomás busca ese usuario en la base y lo deja en `ctx.state.usuario`.

2. Qué número de jugador eres en esa partida: en el servicio de despliegue busco tu participación en la partida:

```
const jugador = await JugadorPartida.findOne({ where: { partidaId, usuarioId: usuario.id } });
```

Esa fila tiene tu `numeroJugador`, que Tomás asigna cuando entras a la partida:

- **Partida privada:** el dueño es el 1 y el invitado es el 2.
- **Matchmaking:** el primero que entró a la cola es el 1 y el segundo es el 2.

3. Qué filas le tocan a ese número: está en config/reglas.js, según la regla 1:

```
const ZONAS_DESPLIEGUE = {
  1: { desde: 1, hasta: 4 },
  2: { desde: 7, hasta: 10 }};
```

4. La revisión de cada barco: `enZonaDespliegue(barco.casilla, jugador.numeroJugador)` comprueba que la fila (`y`) de cada barco caiga dentro de esas filas. Si alguno no cae, el servidor responde `DESPLIEGUE_INVALIDO` con un mensaje como *"LANCHA ... ir en tu zona, entre las filas 7 y 10"*. Eso fue lo que vimos en la prueba con demo_beto.

Por qué se hace así: si el cliente mandara su zona o su número de jugador, cualquiera podría mandar "soy el jugador 1" y desplegar donde quisiera. Como el servidor lo saca de la base, no hay forma de cambiarlo desde el navegador.

Hay dos detalles a considerar:

- **En la E3 sí se puede falsear quién eres**, porque cualquiera puede escribir otro número en el header `X-Usuario-Id`. Eso se corrige en la E4 con JWT, cuando la identidad venga firmada por el servidor.
- **El frontend todavía tiene la zona escrita a mano:** `Preparacion.jsx` usa siempre las filas 1–4. Cuando Vicente conecte esa pantalla, tiene que leer tu `numeroJugador` desde el servidor para pintar la zona correcta. Ese dato va a venir en el `GET /games/:id` que hacemos en el paso 6.

¿Cómo funciona el límite de 2 minutos si el servidor no tiene temporizadores?

Funciona como el plazo de devolución de un libro en la biblioteca: nadie se queda mirando el reloj. La fecha límite queda anotada y se revisa recién cuando alguien llega. El servidor no cuenta los 2 minutos: **guarda la hora límite y la compara cada vez que alguien toca la partida.**

1. Al empezar el despliegue se anota la hora límite. En `iniciarDespliegue`:

```
await partida.update({ expiraEn: new Date(Date.now() + SEGUNDOS_DESPLIEGUE * 1000) });
```

Si la partida pasa a DESPLIEGUE a las 15:00:00, queda guardado `expiraEn = 15:02:00`. No queda corriendo ningún reloj: es solo una fecha en la base.

2. Cada vez que alguien toca la partida, se revisa esa hora. Lo primero que hace `desplegar` es llamar a `resolverDespliegueVencido`:

```
if (partida.estado !== 'DESPLIEGUE' || partida.expiraEn > new Date()) return false;
```

Si todavía no se pasa la hora, no hace nada y el despliegue sigue normal.

3. Si ya pasó, el servidor hace lo que dice la regla 4. Por cada jugador que todavía no está `listo`:

- toma las casillas de su zona con `casillasDeZona`, las mezcla con `crypto` y elige 4;
- crea sus barcos ahí;
- lo marca como listo.

Después llama a `iniciarPartida`, que pasa la partida a EN_CURSO y sortea quién parte.

¿Y si nadie hace nada después de los 2 minutos? La partida sigue en DESPLIEGUE en la base, pero nadie lo nota. La próxima petición que llegue, sea de cualquiera de los dos jugadores, primero resuelve el vencimiento y después responde. Para los jugadores el resultado es el mismo que si el servidor lo hubiera hecho justo a los 2:00. En el paso 6 voy a agregar la misma revisión al `GET /games/:id`, así basta con que alguno de los dos mire la partida para que se resuelva.

Por qué corre en una transacción aparte: si el que llega tarde es quien dispara la revisión, su petición termina con un 409. Si la revisión estuviera en la misma transacción, ese error deshacería también el despliegue al azar. Al separarla, el despliegue al azar queda guardado aunque la petición termine en error.

Es el mismo patrón que ya usa el equipo: Tomás expira los lobbies a los 5 minutos de la misma forma, con `lobbyExpirado` y el mismo campo `expiraEn`.

Limitación: el jugador que espera no recibe un aviso exacto a los 2:00. Se entera cuando su pantalla vuelve a consultar al servidor. Por eso el frontend tiene que preguntar cada pocos segundos. En la E4, con WebSocket, el servidor sí podría avisarle en el momento con un temporizador.

¿Qué pasaría si los dos jugadores despliegan exactamente al mismo tiempo? ¿Qué es el lock y para qué sirve?

Se produciría lo que se llama una **condición de carrera**: dos procesos que leen y escriben los mismos datos al mismo tiempo, y el resultado depende de cuál llega primero.

Sin lock: qué podía salir mal

Cada despliegue hace, dentro de su transacción: crear mis barcos → marcarme `listo` → revisar "¿están los dos listos?" → si es así, iniciar la partida.

Si Karla y Diego despliegan en el mismo instante, las dos transacciones corren en paralelo:

```
Karla                                  Diego
crea sus barcos                        crea sus barcos
se marca listo (sin confirmar aún)     se marca listo (sin confirmar aún)
¿Diego listo?  -> NO lo ve             ¿Karla listo?  -> NO la ve
no inicia la partida                   no inicia la partida
COMMIT                                 COMMIT
```

PostgreSQL no deja ver los cambios de una transacción hasta que esta hace COMMIT. Por eso ninguna ve el `listo` de la otra. Resultado: **los dos quedan listos, pero la partida nunca parte**. Se quedaría en DESPLIEGUE hasta que venza el plazo.

Con otra combinación de tiempos podía pasar lo contrario: que **las dos** iniciaran la partida. Eso significa dos sorteos del primer turno y el +2 de combustible entregado dos veces.

Qué es el lock

Qué es el lock

En `desplegar`, la partida se lee así:

```
const partida = await Partida.findByPk(partidaId, { lock: transaction.LOCK.UPDATE, transaction
});
```

Eso se traduce en SQL a `SELECT ... FOR UPDATE`: **un candado sobre esa fila de la partida**. Funciona como la llave del baño de un local: mientras la tiene uno, el otro espera afuera.

```
Karla                                   Diego
toma el candado de la partida 3         quiere el candado -> ESPERA
crea barcos, se marca listo
¿Diego listo? -> no
COMMIT (suelta el candado)        ->    obtiene el candado
                                        lee datos frescos: Karla ya está listo
                                        crea barcos, se marca listo
                                        ¿Karla listo? -> SÍ -> inicia la partida
                                        COMMIT
```

Los despliegues de una misma partida quedan **en fila, uno detrás del otro**, y el segundo siempre ve lo que hizo el primero. La partida parte **exactamente una vez**.

Lo que mostraron las pruebas

Lancé los dos despliegues en paralelo 3 veces. Las 3 veces pasó lo mismo:

- una respuesta dijo `DESPLIEGUE`, la del primero, que quedó esperando;
- la otra dijo `EN_CURSO`, la del segundo, que hizo part ↓ artida;
- hubo un solo +2 de combustible;
- quedaron exactamente 8 barcos.

Otros detalles

- **Doble clic del mismo jugador:** la segunda petición espera el candado, ve que ya estás `listo` y responde `YA_DESPLEGASTE`. Además, si algo fallara, el `unique (jugadorPartidaId, tipo)` de la tabla impide crear barcos repetidos.
- **Costo:** solo esperan las peticiones a **la misma partida**, y por unos pocos milisegundos. Las demás partidas no se enteran. El candado se suelta solo con el COMMIT, o con el ROLLBACK si hay un error.
- **Tomás usa lo mismo:** en `unirsePorCodigo`, para que dos invitados no entren al mismo lobby, y en `buscarRival`, con `skipLocked`, para que dos jugadores no tomen el mismo rival de la cola.

¿Cómo está funcionando Claude en esta sesión? Mal Regular Bueno ×

¿Por qué el GET responde distinto según el estado de la partida? ¿Qué pasa en el lobby y qué pasa si consulta un administrador?

Porque es **un solo endpoint con dos usos**, y eso lo acordaron en el grupo (el punto 4 que respondió Vicente). El protocolo define un único `GET /games/:id` para reconstruir la pantalla al recargar, y lo que hay que mostrar cambia mucho según la fase de la partida.

El código, en el controlador de Tomás

```
const partida = await partidaService.obtenerPartida(ctx.params.id, ctx.state.usuario);
const estado = await estadoService.armarEstado(partida.id, ctx.state.usuario.id);

ctx.body = estado || detallePartida(partida);
```

1. `obtenerPartida` **de Tomás** revisa los permisos. Si la partida no existe, responde 404. Si no eres jugador **ni administrador**, responde 403.
2. `armarEstado`**, tuyo,** intenta armar la vista de juego. Devuelve `null` en dos casos:
   - la partida no está en DESPLIEGUE, EN_CURSO ni TERMINADA, o sea, sigue en lobby;
   - quien consulta no es uno de los dos jugadores.
3. **Si devolvió** `null`, se responde lo de Tomás (`detallePartida`). Si no, se responde tu estado filtrado.

En el lobby (ESPERANDO_JUGADOR o LISTO_PARA...IAR)

Todavía no hay tablero ni barcos, así que el estado de juego no tendría nada que mostrar. Lo útil en esa fase es lo que arma Tomás:

```
{ "partidaId": 1, "estado": "ESPERANDO_JUGADOR", "codigoInvitacion": "K7P2QX",
  "expiraEn": "...", "jugadores": [{ "id": 1, "username": "demo_dueno", "rolPartida":
"DUENO", "numeroJugador": 1 }] }
```

Con eso la pantalla del lobby muestra el código para invitar, quién está y cuánto queda antes de que expire.

Si consulta un administrador

Tomás deja que un administrador vea cualquier partida, aunque no juegue en ella, porque la Propuesta dice que el admin **puede ver las partidas pero no jugar**. `armarEstado` no encuentra al admin entre los jugadores, así que no puede armar un "yo": la vista filtrada siempre se arma desde los ojos de un jugador. Por eso devuelve `null`, y el admin recibe el resumen de Tomás: estado, ronda y jugadores, **sin tablero ni barcos.**

Esto está bien por ahora, porque al admin no le corresponde ver la vista privada de ningún jugador. Si en la E5 el panel de administración necesita auditar el tablero completo, se hace con una ruta aparte, protegida por rol, y no mezclándolo con la vista de juego.

Lo que tiene que saber el frontend

Las dos respuestas tienen forma distinta, pero ambas traen `partidaId` y `estado`. El frontend mira `estado` para saber qué forma le llegó: si es lobby, usa `jugadores` y `codigoInvitacion`, y si es juego, usa `yo`, `rival`, `casillasVisibles`, etc. Conviene anotarlo en el README cuando documentemos el endpoint.

¿Por qué el `GET` revisa primero si venció el despliegue?

Porque el plazo de 2 minutos no lo vigila ningún reloj del servidor. Se resuelve **cuando alguien toca la partida**, y el `GET` es justamente lo que más se va a llamar.

El problema si solo lo revisara `desplegar`

Imagina este caso:

1. Ana despliega a tiempo y queda esperando.
2. Beto cierra el navegador y nunca despliega.
3. La pantalla de Ana consulta el estado cada pocos segundos con `GET /games/:id`.

Si el vencimiento se revisara solo en `desplegar`, nadie lo dispararía nunca. Beto no va a llamar a nada, y Ana ya desplegó, así que su pantalla no vuelve a llamar a `desplegar`. Ana se quedaría viendo "esperando al rival" para siempre, que es justo lo que la regla 4 quiere evitar.

Al revisarlo en el `GET`, la primera consulta de Ana después de los 2 minutos hace que el servidor ubique los barcos de Beto al azar y arranque la partida. Esa misma respuesta ya le llega a Ana con `estado: "EN_CURSO"`.

Por qué va *primero*

```
async function armarEstado(partidaId, usuarioId) {
  await resolverDespliegueVencido(partidaId);
  const partida = await Partida.findByPk(partidaId);
  ...
```

Si el estado se armara antes de resolver el vencimiento, Ana recibiría una respuesta desactualizada: diría DESPLIEGUE, aunque en la base la partida ya habría pasado a EN_CURSO. Recién en la consulta siguiente vería el cambio. Al resolver primero y leer después, **la respuesta siempre refleja la situación real en ese momento**.

Detalles que lo hacen seguro

- **Si no venció, no hace nada:** solo revisa la hora y sigue. No cambia la respuesta ni la hace notoriamente más lenta.
- **Si los dos consultan justo al vencer, se resuelve una sola vez.** `resolverDespliegueVencido` bloquea la fila de la partida (el `lock` que vimos) y vuelve a mirar el estado dentro del bloqueo. El segundo ya ve EN_CURSO y no repite nada.
- **Usa su propia transacción:** el despliegue al azar queda guardado aunque algo posterior falle.

Esto lo probamos en el paso 6: dejé el plazo vencido en la partida 3, consulté con el `GET` y la respuesta ya vino con la partida EN_CURSO y los 4 barcos del rival ubicados.

¿Qué campos quedaron en null o pendientes y de quién dependen? (eventoRonda, resultadoFinal, contacto perdido)

Son tres campos del estado que quedaron vacíos, más algunas reglas de mover que dependen de piezas que todavía no existen.

Campos del estado

Campo	Qué debería mostrar	Por qué está vacío	Depende de
`eventoRonda` (`null`)	La carta de la ronda: Tormenta, Botín flotante, Sabotaje o Marea alta (regla 33)	No existe la tabla `evento_partida` y nadie revela cartas. La carta se saca al empezar cada ronda, y eso pasa cuando cambia el turno	Quien tome `evento_partida` (sin dueño) y el **terminar turno de Tomás**, que es donde cambia la ronda
`resultadoFinal` (`null`)	Ganador, motivo y puntaje detallado al terminar (reglas 35 a 37)	Nadie calcula todavía el fin de la partida: flota destruida, abandono o ronda 20	**Tomás** (abandonar y llegar a la ronda 20) y quien haga el **puntaje**. Es parte de "victoria/derrota" en la E4
Contacto perdido (`visible: false` con `vistoEnRonda`)	La última posición conocida de un enemigo que salió de tu radio (regla 20)	Para recordarla hay que guardar cada avistamiento, y hoy no hay dónde: no está en el ER. Por ahora solo se ven los enemigos que están en tu radio en este momento	**Tú**, porque la visibilidad es tuya. Necesita una tabla nueva, por ejemplo `Avistamientos`, y anotarla en el README como ajuste del ER

Campos con valor fijo o que no se aplican

Campo	Situación	Depende de
`conectado` siempre `true` y `segundosParaAbandono` siempre `null`	Saber si el rival está conectado requiere WebSocket	La **E4** (WebSocket)
`turnoExpiraEn`	Muestra la hora, pero nadie cierra el turno cuando vence	El **terminar turno y el timeout de Tomás** (regla 15)

Reglas de mover que quedaron pendientes

Regla	Qué falta	Depende de
Combate cuerpo a cuerpo (26, 27, 40, 42)	Cuando dos barcos enemigos quedan adyacentes, hoy no pasa nada	**Tú**, con la tabla `Combates` que te propusiste. Es el pendiente más importante

Chatarra al hundirse (28, 41)	Un barco hundido debería dejar 2 de chatarra en su casilla. Recoger chatarra ya funciona, pero nadie la genera	**Tú**, junto con el combate
Marea alta	Esa ronda las corrientes no deberían dañar	La tabla de eventos
Recolección y combustible de reserva (8, 18)	Al empezar tu turno, cobrar los pozos y depósitos ocupados y darte 1 de combustible si quedaste en 0	El **inicio de turno de Tomás**

En resumen: de lo pendiente, **a ti te tocan el combate, la chatarra al hundirse y el contacto perdido**. El resto depende del turno de Tomás, de quien tome los eventos y de los WebSocket de la E4. Conviene anotar todo esto en el README como pendiente, para que en la corrección quede claro que es una decisión de alcance y no algo olvidado.

¿En qué orden se validan las jugadas y por qué ese orden?

Las jugadas se validan **de lo general a lo particular**, y el servidor **se detiene en el primer error, sin haber tocado nada**. Es el orden del punto 7 del protocolo y del punto 2.11 de la Propuesta.

El orden en tu código

#	Qué revisa	Error si falla
0	**El header:** quién eres (middleware de Tomás)	401 `USUARIO_NO_IDENTIFICADO`
1	**El formato:** id de partida válido, `tipoAccion` conocido, que vengan `barcoId` y `casillaDestino`. Se revisa sin consultar la base	400 `ID_INVALIDO`, `TIPO_ACCION_INVALIDO`, `CAMPOS_FALTANTES` / 422 `ACCION_NO_DISPONIBLE`
2	**La partida:** existe y eres jugador de ella	404 `PARTIDA_NO_EXISTE` / 403 `NO_PERTENECES_A_LA_PARTIDA`
3	**Partida activa:** está EN_CURSO	422 `PARTIDA_NO_ACTIVA`
4	**Tu turno**	422 `NO_ES_TU_TURNO`
5	**El barco:** existe, es tuyo y está a flo	404 `BARCO_NO_EXISTE` / 403 `BARCO_NO_ES_TUYO` / 422 `BARCO_HUNDIDO`

6	**Acciones:** te quedan, y ese barco no usó ya 2	422 `SIN_ACCIONES` / `LIMITE_ACCIONES_BARCO`
7	**Recursos:** combustible para al menos una casilla (regla 18)	422 `COMBUSTIBLE_INSUFICIENTE`
8	**El destino:** dentro del tablero, distinto de donde estás, dentro del movimiento, combustible para todo el trayecto, sin un barco visible y con algún camino libre	400 `FUERA_DEL_TABLERO` / 422 `DESTINO_INVALIDO`, `FUERA_DE_MOVIMIENTO`, `COMBUSTIBLE_INSUFICIENTE`, `CASILLA_OCUPADA`, `RUTA_BLOQUEADA`
✓	Recién aquí **se aplica el movimiento**: mover el barco, cobrar el combustible, restar la acción y guardarla	200

Por qué ese orden

1. El error que recibes es el que de verdad importa. Si no es tu turno, no tiene sentido decirte "te falta combustible": aunque lo tuvieras, igual no podrías jugar. Cada paso solo tiene sentido si el anterior pasó. Por ejemplo, no se puede revisar si "ese barco ya usó 2 acciones" sin saber antes que el barco existe y es tuyo.

2. Primero lo barato, al final lo caro. El formato se revisa sin tocar la base. Lo más costoso queda al final: cargar los barcos de la partida, calcular qué ves y buscar el camino. Así, una petición mal hecha se rechaza rápido, sin gastar consultas.

3. No se le da información a quien no le corresponde. Revisar que eres jugador (paso 2) antes que cualquier cosa del juego evita que alguien de afuera pruebe ids de barcos o averigüe en qué estado va la partida. Lo mismo con el turno: el rival no puede tantear tus barcos mientras juegas.

3. No se le da información a quien no le corresponde. Revisar que eres jugador (paso 2) antes que cualquier cosa del juego evita que alguien de afuera pruebe ids de barcos o averigüe en qué estado va la partida. Lo mismo con el turno: el rival no puede tantear tus barcos mientras juegas.

4. Nada queda a medias. Ninguna validación modifica nada: todas solo leen. Los cambios se hacen recién cuando todo pasó, y además todo corre en una transacción. Si algo falla en la mitad de la aplicación, se deshace completo. Por eso en la prueba ninguno de los 17 errores gastó acciones ni combustible, que es lo que exige la regla 39.

5. El equipo y el frontend saben qué esperar. Como sigue el orden escrito en el protocolo, Vicente puede mostrar los mensajes sabiendo cuál error aparece primero, y las demás acciones (disparar, vigía, mejorar) van a seguir la misma secuencia.

Un detalle del paso 7: el combustible se revisa en dos momentos. Primero se mira si alcanza para **una** casilla (regla 18, antes del destino, como dice el protocolo). Después, cuando ya se conoce la distancia, se mira si alcanza para el **trayecto completo**. El costo total no se puede calcular antes de validar el destino.

¿Por qué el tablero de la demo es fijo y no se genera al azar con generarCasillas()?

Porque un seeder tiene que dar **siempre el mismo resultado**, y en tu demo hay muchas cosas que dependen de dónde está cada casilla especial.

1. El enunciado pide datos reproducibles. La E3 pide seeders que inserten *"datos de prueba reproducibles"*. Con un tablero fijo, cada `npm run db:reset` deja exactamente la misma partida, tanto en tu computador como en el de Tomás, el de Vicente o el del ayudante que corrige.

2. El resto de los datos del seeder depende del tablero. Tu seeder no solo crea casillas, también arma una historia coherente encima:

- Karla tiene **revelada la corriente de (2,3)**, así que esa casilla tiene que ser una corriente.
- La acción de historial de la ronda 2 dice que la fragata **se frenó en esa corriente** y perdió 1 de casco, y por eso la fragata aparece con casco 1/2.
- La lancha de Karla está sobre el **pozo de (5,4)**, así que el estado muestra `ocupadaPor` en esa casilla.

Si el tablero fuera al azar, la corriente podría salir en otra parte. Karla tendría "revelada" una casilla de agua y la historia no cuadraría. Incluso podría caer una especial justo debajo de un barco o donde no conviene.

3. La colección de Postman y la guía del README cuentan con esas posiciones. Por ejemplo:

- "Mover el acorazado de (4,2) a (4,3)" necesita que (4,3) sea agua libre.
- "Mover a una casilla con un barco visible" usa la lancha de Diego en (6,6), que tiene que estar dentro del radio de Karla.
- La guía dice que Karla ve la corriente revelada y el pozo.

- La guía dice que Karla ve la corriente revelada y el pozo.

Con un tablero al azar, esas peticiones darían resultados distintos cada vez y los tests fallarían de vez en cuando.

4. Igual respeta las reglas. El tablero fijo cumple la regla 7: 3 casillas de cada especial, con pozos, depósitos y naufragios solo entre las filas 4 y 7.

¿Y la regla 34, que dice que el azar sale del servidor? Sigue cumpliéndose en las **partidas reales**. Cuando una partida nueva pasa a despliegue, `generarTablero` llama a `generarCasillas()` y sortea el mapa con `crypto`. El tablero fijo es solo para la **partida de demostración** del seeder, que existe para probar y mostrar, no para jugar en serio. Tomás hizo lo mismo con el tablero de la partida terminada.

En resumen: el azar va en el juego real y la repetibilidad va en los datos de prueba.

¿Por qué el seeder tiene que correr después del 002 y del 004? ¿Qué pasaría si corriera antes?

Porque tu seeder **usa datos que crean esos dos**. Sin ellos no tiene sobre qué construir.

De qué depende

Seeder	Lo que crea	Para qué lo necesita tu 005
002 (Tomás)	Los usuarios `demo_*`, la partida en curso y los `JugadoresPartida` de Karla y Diego	Las casillas apuntan a la partida (`partidaId`) y los barcos apuntan al jugador (`jugadorPartidaId`). Sin esas filas, las FK no tienen a qué apuntar
004 (tuyo)	Las 4 mejoras del catálogo	La mejora del destructor de Karla apunta a la fila `BLINDAJE` del catálogo (`catalogoMejoraId`)

Cómo se asegura el orden: `sequelize-cli` corre los seeders en orden alfabético por nombre de archivo, igual que las migraciones. Por eso los números importan: `002` → `004` → `005`. Además, antes de los seeders tienen que haber corrido todas las migraciones, porque las tablas tienen que existir. Eso ya lo hace `db:reset` en ese orden.

Qué pasaría si corriera antes

Antes del 002: tu seeder parte buscando a Karla y Diego en una partida en curso. Como todavía no existen, la búsqueda no encuentra nada y el seeder termina sin hacer nada:

```
const demo = await buscarPartidaDemo(queryInter );
if (!demo) return;
```

Esa revisión evita que se caiga con un error de FK, pero deja un problema escondido. Sequelize anota el seeder como ejecutado en `SequelizeData`, así que, cuando después corre el 002 y crea la partida, tu seeder **no vuelve a correr**. La partida demo queda sin tablero ni barcos, y nadie ve un error que avise.

Después del 002, pero antes del 004: la búsqueda de `BLINDAJE` no encuentra la fila, así que la mejora no se inserta (también está protegido con un `if`). Pero el destructor de Karla igual se crea con casco 4/4, porque ese valor está escrito en el seeder. Quedaría un barco con más casco del normal y **sin la mejora que lo justifica**, más una acción de historial que dice que compró Blindaje. Los datos quedarían incoherentes.

En resumen

El número del archivo es lo que garantiza que tu seeder encuentre todo lo que necesita. Si alguien lo renombrara a `001-...`, por ejemplo, la demo se rompería sin ningún error visible. Por eso los seeders, igual que las migraciones, se numeran pensando en qué depende de qué.

¿Qué pasa en la pantalla cuando el servidor rechaza una jugada?

La pantalla **muestra el mensaje que manda el servidor y deja todo como estaba**, porque el servidor no aplicó nada.

El recorrido de una jugada rechazada

1. **Haces clic en el destino** y `mover()` manda la jugada:

```
const { datos } = await pedirApi(`/games/${partidaId}/acciones`, { metodo:
"POST", usuarioId, cuerpo: {...} });
```

2. **El servidor la rechaza**, por ejemplo con `422` y `{ "codigo": "FUERA_DE_MOVIMIENTO", "mensaje": "El destino esta a 5 casillas y este barco mueve 2." }`. Por la regla 39, el servidor no tocó nada: ni el barco, ni el combustible, ni las acciones.
3. **El helper `pedirApi` de Tomás convierte la respuesta en un error.** Como la respuesta no fue exitosa, lanza un `Error` con el `mensaje` del servidor, y le agrega el `status` y el `codigo`.
4. **`mover()` atrapa ese error y lo muestra:**

```
} catch (error) {
  setMensaje(error.message);
} finally {
  setEnviando(false);
}
```

   El mensaje aparece debajo del tablero. Eso fue l … e vimos en la prueba con la fragata: *"El destino esta a 5 casillas y este barco mueve 2."*

5. **No se recarga nada a propósito.** Cuando hay error no se llama a `cargar()`, porque el estado no cambió. El barco sigue seleccionado, así que puedes elegir otro destino de inmediato, y la consulta cada 2 segundos sigue funcionando igual.

`setEnviando(false)` va en el `finally`, así que pase lo que pase se vuelven a habilitar los clics y los botones.

Algunos errores se avisan antes de preguntar al servidor

Para no hacer una petición que se sabe que va a fallar, `handleCasilla` revisa algunas cosas en el navegador y muestra el mensaje directo:

- *"No es tu turno, espera a que el rival termine."*
- *"Selecciona primero uno de tus barcos."*
- *"Ya usaste todas las acciones del turno."*
- *"Ese barco ya usó sus 2 acciones de este turno."*

Son solo para responder más rápido. Aunque alguien se las saltara modificando el JavaScript, el servidor las vuelve a revisar y rechaza la jugada igual.

Lo mismo vale para los otros botones

- **Terminar turno** y **Abandonar** usan el mismo patrón: si el servidor responde con error, se muestra su mensaje y no cambia nada.
- **Si se cae la conexión con el servidor**, `pedirApi` lanza *"No se pudo conectar con el servidor."*. Si ya había una partida cargada, el mensaje aparece debajo del tablero y se sigue viendo lo último que llegó. Si todavía no había cargado nada, la panta… muestra el error con el botón "Volver al menú".

¿Qué pasa al apretar “Terminar turno”? ¿Cómo se entera la pantalla del rival de que ahora le toca

Pasan dos cosas en momentos distintos: el servidor cambia el turno en la base, y la pantalla del rival se entera en la siguiente consulta, a lo más 2 segundos después.

1. En tu pantalla, al apretar el botón

El botón solo se habilita si la partida está en curso, es tu turno y no hay otra petición en camino:

```
disabled={!enCurso || !esMiTurno || enviando}
```

Al apretarlo, `terminarTurno()` llama al endpoint de Tomás:

```
await pedirApi(`/games/${partidaId}/terminar-turno`, { metodo: "POST", usuarioId });
```

2. En el servidor (`turno.service` de Tomás)

1. **Valida:** la partida tiene que estar en curso y tiene que ser tu turno. Si no, responde 422 (`PARTIDA_NO_ACTIVA` o `NO_ES_TU_TURNO`).
2. **Pasa el turno al otro jugador:**
   - **Si el turno vuelve al jugador que partió la partida, se acaba la ronda:** la ronda sube en 1 y se revela una carta de evento nueva (regla [illegible]). Si se pasara de la ronda 20, la partida termina por límite de rondas y se calculan los puntajes.

- **Empieza el turno del rival:** recolecta lo que dan las casillas de recurso que ocupa (regla 8), recupera sus 3 acciones (regla 14) y recibe 1 de combustible de reserva si quedó en 0 y sin casillas de recurso (regla 18).
- **Se guarda en la partida** el nuevo `turnoActualId` y un `expiraEn` con 3 minutos más (regla 15).

3. **Responde** con un resumen: `{ partidaId, estado, ronda, jugadorActivo, turnoExpiraEn }`.

Todo esto ocurre en una transacción: o se aplica completo, o no se aplica nada.

3. De vuelta en tu pantalla

```
setSeleccionado(null);
setMensaje(`Terminaste tu turno. Ahora juega ${estado.rival.username}.`);
await cargar();
```

Se deselecciona el barco, aparece el mensaje y se recarga el estado. Tu panel pasa a decir "**Turno del rival**" y los botones de jugar quedan deshabilitados.

4. Cómo se entera la pantalla del rival

Consultando cada 2 segundos. En la E3 no hay WebSocket, así que el servidor no tiene forma de avisarle al rival. Su pantalla de Partida pregunta sola:

```
const consulta = setInterval(cargar, MILISEGUNDOS_CONSULTA);
```

En la siguiente consulta, que llega a lo más 2 segundos después, el `GET /games/:id` ya trae `jugadorActivo` con su id. La pantalla se actualiza so[illegible] recargar:

- el panel dice "**Es tu turno**";
- vuelve a tener **3 acciones disponibles**;
- los recursos aparecen con lo que recolectó;
- si empezó una ronda nueva, se ve la **carta de evento** y el número de ronda;
- el reloj de 3 minutos empieza a correr según `turnoExpiraEn`.

Fue lo que vimos en la prueba: Karla terminó su turno y la pantalla de Diego cambió sola. Cuando Diego terminó el suyo, apareció la ronda 5 con una carta nueva.

Si nadie aprieta el botón

Si se acaban los 3 minutos, el turno se cierra igual. Funciona como el plazo del despliegue: no hay un reloj en el servidor, sino que **el `GET` revisa si el turno venció** (`resolverTurnoVencido`, de Tomás) y, si venció, pasa el turno. Como las dos pantallas consultan cada 2 segundos, eso ocurre casi justo al vencer.

Qué cambia en la E4

Con WebSocket, el servidor le **mandará el estado al rival en el mismo momento** en que se termina el turno, en vez de esperar a que la pantalla pregunte. Así se evitan las consultas cada 2 segundos y el cambio se ve al instante. El enunciado de la E4 pone justo esto como ejemplo de evento en tiempo real.

Hace 1 minuto